Faculty of Computing and Artificial Intelligence
Department of Software Engineering

AI-Assisted Verification of Source Code Against Functional Requirements

“EzzyHub”

Supervised by

Dr. Soha Makady

Assisted by

TA. Hassan Mourad

Implemented by

20227005	*Amira Ahmed Ibrahim*
20226017	*Amira Emad Fouad*
20226026	*Basant Hatem Hassan*
20226122	*Basmala Khalifa Mohamed*
20226041	*Raghad Ahmed Nabil*

Graduation Project

Academic Year 2025-2026

Phase 1 / Final Documentation

Table of contents

Chapter 1: Introduction 1

1.1 Background and Motivation 1

1.2 Problem Definition 2

1.3 Project Objectives and Proposed Solution 3

1.4 Project Scope and limitations 3

1.5 Development Methodology 4

1.6 Tools and Technologies Used (Software and Hardware) 6

1.7 Project Timeline / Gantt Chart 7

1.8 Report Organization 7

Chapter 2: Related Work 8

2.1 Overview 8

2.2 Comparative analysis: What distinguishes this project from existing work? 8

2.3 All sources properly cited using IEEE format 9

Chapter 3: System Analysis 10

3.1 Stakeholder Analysis 10

3.2 Functional Requirements 11

3.3 Non-Functional Requirements 15

3.4 Use Case 16

3.5 Feasibility Study (technical, economic, and operational - brief) 18

Chapter 4: System Design 19

4.1 System Architecture 19

4.2 Class Diagrams 20

4.3 Sequence Diagrams (for key use cases) 26

4.4 Database Entity-Relationship Diagram (ERD) 31

4.5 User Interface Wireframes / Mockups 32

4.6 Security Design Considerations 33

Chapter 5: Implementation 39

5.1 Implementation Environment and SetupEnvironment: 39

5.2 Key Implementation Details (algorithms, modules, major decisions) 39

5.3 Sample Screenshots / Output Samples 47

Chapter 6: Testing and Evaluation 53
6.1 Testing Strategy 53
6.2 Test Cases and Results (table format) 56
6.3 Performance Evaluation / Benchmarking (where applicable) 63
6.4 Limitations and Known Issues 66
Chapter 7: Conclusion and Future Work 69
7.1 Summary of Achievements 69
7.2 Lessons Learned 70

List of Figures

Figure 1: EzzyHub Verification Workflow 4
Figure 2: Project Development Gantt Chart 5
Figure 3: Use Case Diagram of the EzzyHub Platform 6
Figure 4: EzzyHub System Architecture 16
Figure 5: System Architecture Diagram
Component Diagram 18
Figure 6: System Component Diagram 19
Figure 7: Class Diagram Requirement Analysis 20
Figure 8: Class Diagram Static Analysis (Sonar) 21
Figure 9: Class Diagram Static Analysis (Sonar) 22
Figure 10: Class Diagram Code Analyzer 23
Figure 11: Class Diagram AI Suggestions 24
Figure 12: Sequence Diagram Requirement Analysis 25
Figure 13: Sequence Diagram Static Analysis (Sonar) 26
Figure 14: Sequence Diagram AI Suggestions 29
Figure 15: Database Entity-Relationship Diagram (ERD) 30
Figure 16: User Interface Wireframes 33
Figure 17: Sign In Screen 46
Figure 18: Sign Up Screen 47
Figure 19: Code Analyzer Screens 49
Figure 20: Requirement-Code Matching Screens 51
Figure 21: Sonar Analyzer Screens 52
Figure 22: AI Suggestions Screens 53
Figure 23: Report History Screen 54

List of Tables

Table 1: Tools and Technologies Used 6
Table 2: Comparison of EzzyHub with Existing Tools 8
Table 3: Requirement Analyzer Unit and Integration Test Summary 58
Table 4: Requirement Analyzer API / Cross-Service Test Cases (Postman) 59
Table 5: Sonar Analyzer Unit and Controller Test Summary 59
Table 6: Sonar Analyzer API / Integration Test Cases (Postman) 60
Table 7: Matching Service Unit and Integration Test Summary 60
Table 8: Matching Service Representative Test Cases 62
Table 9: Code Analyzer Unit and Integration Test Summary 63
Table 10: Code Analyzer API / Cross-Service Test Cases (Postman) 63
Table 11: Single-Requirement Latency 64
Table 12: Throughput Under Concurrent Load 64
Table 13: Trace-Link Retrieval vs. Published Baseline (iTrust) 65
Table 14: Final Status Classification vs. Oracle (131 Requirements) 65

List of Abreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
CI/CD	Continuous Integration / Continuous Delivery
CORS	Cross-Origin Resource Sharing
CVE	Common Vulnerabilities and Exposures
ERD	Entity-Relationship Diagram
FR	Functional Requirement
HTTP	Hypertext Transfer Protocol
IDC	Incremental Development Cycle
IR	Information Retrieval
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning

NFR	Non-Functional Requirement
NLP	Natural Language Processing
NLTK	Natural Language Toolkit
POS	Part of Speech
QA	Quality Assurance
RAM	Random Access Memory
REST	Representational State Transfer
SBERT	Sentence-BERT
SQL	Structured Query Language
SSRF	Server-Side Request Forgery
UI	User Interface
VSM	Vector Space Model
XSS	Cross-Site Scripting

Abstract

EzzyHub: An AI-Assisted Microservices Platform for Automated Source Code Verification Against Functional Requirements

Verifying that a software implementation faithfully satisfies its specified functional requirements is a fundamental yet labor-intensive and error-prone activity in software engineering, traditionally performed through manual inspection and review. This project introduces **EzzyHub**, an AI-assisted platform built on a microservices architecture that automates the verification of uploaded source code against a defined set of functional requirements. The system decomposes the verification workflow into independent, loosely-coupled services—covering requirement analysis, code analysis, semantic matching, static executability verification, and AI-driven improvement suggestions—coordinated through a central matching service that acts as the orchestrator. Natural Language Processing techniques are used to extract and structure the functional requirements, while the submitted codebase is parsed to derive comparable representations. A hybrid semantic-matching approach that combines cosine similarity over sentence embeddings with Jaccard similarity over extracted keywords is applied to determine the extent to which each requirement is fulfilled, classifying the outcome as *Implemented*, *Partially Implemented*, or *Missing*. In parallel, a standalone static-analysis stage assesses the code's executability and quality. The platform integrates multiple large language model providers to generate actionable suggestions and produces three complementary reports covering requirement matching, code analysis, and static quality assessment. By automating requirement-to-code traceability, EzzyHub reduces manual verification effort, improves consistency, and delivers explainable, evidence-based insights that support developers, reviewers, and educators.

منصة قائمة على الخدمات المصغّرة ومدعومة بالذكاء الاصطناعي للتحقق الآلي من مطابقة الكود المصدري للمتطلبات الوظيفية :EzzyHub

يُعدّ التحقق من أن تنفيذ البرمجيات يحقق بدقة المتطلبات الوظيفية المحددة له نشاطًا أساسيًا في هندسة البرمجيات، إلا أنه في الوقت ذاته مهمة شاقة ومعرّضة للأخطاء، تُنفَّذ تقليديًا عبر الفحص والمراجعة اليدوية. يقدّم هذا المشروع منصة **EzzyHub**، وهي منصة مدعومة بالذكاء الاصطناعي ومبنية على معمارية الخدمات المصغّرة (Microservices)، تقوم بأتمتة عملية التحقق من مطابقة الكود المصدري المُرفوع لمجموعة محددة من المتطلبات الوظيفية. يُقسّم النظام سير عمل التحقق إلى خدمات مستقلة ومترابطة ترابطًا مرنًا — تشمل تحليل المتطلبات، وتحليل الكود، والمطابقة الدلالية، والتحقق الساكن من قابلية التنفيذ، تُستخدَم تقنيات .(Orchestrator) واقتراحات التحسين المولّدة بالذكاء الاصطناعي — تُنسَّق جميعها من خلال خدمة مطابقة مركزية تعمل كوحدة تنسيق لاستخلاص المتطلبات الوظيفية وهيكلتها، بينما يُحلَّل الكود المُقدَّم لاستخراج تمثيلات قابلة للمقارنة. ويُعتمد أسلوب مطابقة دلالي (NLP) معالجة اللغة الطبيعية على الكلمات المفتاحية (Jaccard Similarity) على التضمينات الجُملية وتشابه جاكارد (Cosine Similarity) هجين يجمع بين تشابه جيب التمام المستخرجة، لتحديد مدى تحقّق كل متطلب وتصنيفه إلى: *مُنفَّذ*، أو *مُنفَّذ جزئيًا*، أو *مفقود*. وبالتوازي، تُقيّم مرحلة تحليل ساكن مستقلة قابلية تنفيذ الكود وجودته.

لتوليد اقتراحات قابلة للتطبيق، وتُنتج ثلاثة تقارير متكاملة تغطي مطابقة المتطلبات، وتحليل الكود، (LLMs) كما تدمج المنصة عدة مزوّدي نماذج لغوية كبيرة الجهد اليدوي في التحقق، وتُحسّن الاتساق، وتوفّر رؤى مفسَّرة وقائمة على EzzyHub وتقييم الجودة الساكنة. وبأتمتة تتبّع المتطلبات وصولًا إلى الكود، تقلّل .الأدلة تدعم المطوّرين والمراجعين والمعلّمين

Chapter 1: Introduction

1.1 Background and Motivation

Modern software systems are continuously growing in size, complexity, and dependency on dynamic development environments. Requirements frequently change, new features are added, and multiple contributors work on the same codebase, making it increasingly difficult to ensure that systems remain executable and aligned with their intended functional requirements. At the same time, AI-assisted coding tools are widely used to accelerate development, but they often generate code with missing imports, unresolved dependencies, configuration errors, or incomplete implementations of required functionality. These issues reduce trust in generated code and increase the effort required for manual verification.

In recent years, Artificial Intelligence (AI) tools have gained popularity in software development, especially for automatic code generation. These tools reduce development time and support developers in implementing features and exploring solutions. However, despite their benefits, AI-generated code introduces challenges related to correctness, reliability, and trustworthiness.

The research question addressed is: *How can an AI-assisted system automatically verify requirement compliance and code executability while generating complete, ready-to-use code solutions efficiently?*

Motivation

A major issue in both AI-generated and human-written code is the missed or partial implementation of functional requirements. Requirement-related defects remain a leading cause of software failure, accounting for about 44% of errors. This unreliability reduces developer confidence, as 76% of developers report lacking trust in AI-generated code and therefore rely on manual verification of requirement compliance, a process that becomes increasingly difficult in large, distributed projects. The impact of manual verification is costly, with developers spending around 20% of their time fixing bugs, approximately $20,000 per developer annually, contributing to a Software Verification Services market valued at $103.2 billion in 2024. AI-generated code also introduces execution-preventing

issues such as missing imports, incomplete configurations, or incorrect dependencies, with 80–90% of generated snippets requiring manual correction before production use. In addition, maintaining traceability between requirements and implementation remains a persistent challenge, with over 65% of projects struggling to preserve these links. Together, these issues highlight the lack of automated mechanisms to verify executability and requirement fulfillment.

1.2 Problem Definition

There currently exists no automated system that systematically verifies whether source code whether generated by AI tools or written by human developers both satisfies explicitly stated functional requirements and is executable in its current form without manual fixes. This problem manifests in several specific challenges:

Problem 1: Functional Requirement Gaps Developers cannot easily determine which functional requirements have been fully implemented, partially implemented, or completely missing from the codebase. This is especially critical in AI-generated code, where the model may misunderstand requirements or generate incomplete solutions.

Problem 2: Execution-Preventing Issues Code may appear logically sound but fail to execute due to missing imports, incorrect package references, or incomplete configuration files. These issues prevent requirement validation entirely, as the code cannot run to demonstrate functionality.

Problem 3: Lack of Trust and Verification Overhead Developers cannot confidently trust AI-generated code for production use, and the manual verification process is so time-intensive that it negates much of the productivity gain from using AI tools. Similarly, human-written code may contain the same issues but lacks systematic verification.

Problem 4: Fragmented Verification Approach Existing verification tools handle syntax, static analysis, or unit testing but lack an integrated system ensuring executability, requirement compliance, and consistency. Without a unified gate confirming code is runnable and compliant, AI coding tools face adoption barriers and development slows, especially in complex projects with many classes, dependencies, and configurations.

1.3 Project Objectives and Proposed Solution

EzzyHub aims to provide an automated system that verifies whether source code whether AI-generated or human-written fulfills its stated functional requirements, while also offering standalone static analysis to surface code-quality and execution issues. Its specific objectives are to:

- Automatically analyze a software project's structure and extract its classes, methods, and code features.
- Accept natural-language functional requirements and process them using an NLP pipeline.
- Match each requirement against the codebase and classify it as Implemented, Partial, or Missing, with a measurable coverage percentage.
- Generate AI-assisted code suggestions to close partial or missing requirements.
- Provide standalone static analysis that detects bugs, vulnerabilities, and code smells.
- Present the results in clear, traceable reports.

Proposed Solution

EzzyHub is a microservices-based platform. Source code is acquired through a ZIP upload and parsed to extract its structure and features. In parallel, functional requirements are processed by a dedicated NLP service that extracts keywords and action verbs and produces semantic embeddings. A matching engine then compares requirements against code using combined semantic (cosine) and keyword (Jaccard) similarity, classifying each requirement and computing overall coverage producing a **matching report**. For partial or missing requirements, an AI suggestion service generates candidate code, producing a **code report** (ready-to-use source files). Independently, a standalone static-analysis service (SonarQubebased) scans the uploaded project for bugs, vulnerabilities, and code smells, producing a **static-analysis report**. These three reports are generated separately.

1.4 Project Scope and limitations

- **Scope**
  - Supports software projects in multiple programming languages (e.g. Java, C++, C#).
  - Accepts code via ZIP upload, and functional requirements as natural-language text.

- Performs requirement-to-code traceability, coverage calculation, and AI-assisted suggestions.
- Provides standalone static analysis (bugs, vulnerabilities, code smells, execution issues).
- Provides a web-based interface and generates downloadable reports.

➢ **Limitations**

- The NLP pipeline is English-only (spaCy English models and a curated English domain synonym map); requirements written in other languages are not supported.
- Semantic validation in requirement analysis depends on an external LLM API; if it is unavailable, the system falls back to a simpler rule-based check.
- Static analysis relies on an external SonarQube instance running separately.
- Due to time constraints, executability verification is provided only through the standalone static-analysis stage, rather than being integrated into the matching and AI-suggestion flow.

1.5 Development Methodology

The project followed an incremental, Agile-style methodology. The system was decomposed into independent microservices (requirement analysis, code analysis, matching, AI suggestions, reporting, authentication, static analysis, and an API gateway), and team members worked on their services in parallel. Each service was developed, tested, and integrated incrementally, allowing features to be built and validated independently before being combined through the API gateway. This service-oriented division supported parallel development, clear ownership, and easier integration, while shared contracts (REST APIs and DTOs) kept the services interoperable.

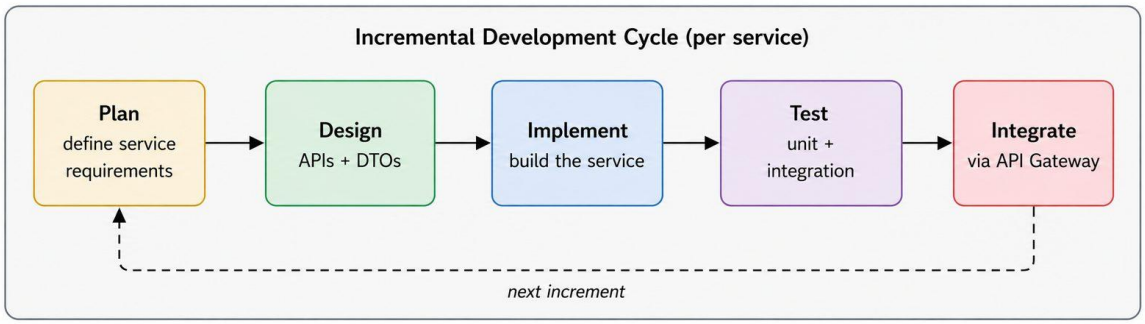


Figure 1: EzzyHub Verification Workflow

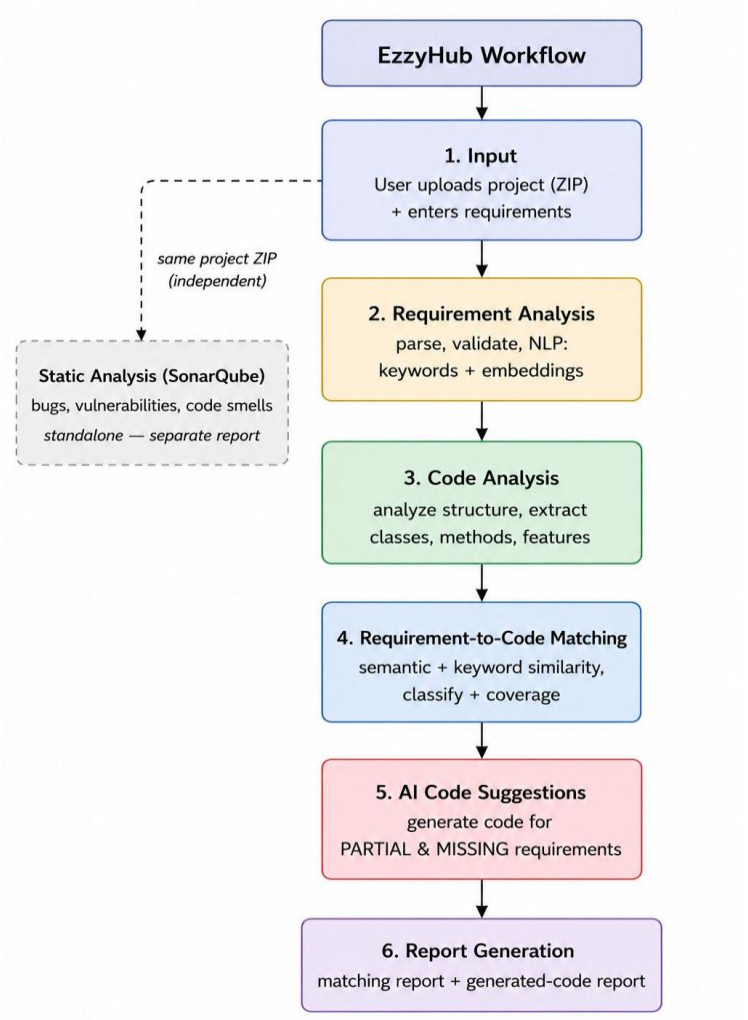


Figure 2: Project Development Gantt Chart

1.6 Tools and Technologies Used (Software and Hardware)

The following tools and technologies were used across the EzzyHub platform, spanning backend services, the NLP/ML pipeline, AI integrations, data infrastructure, and the frontend.

Category	Tool / Technology	Purpose
Backend	Java 17 & 21, Spring Boot 3.2.x	Microservices development
Backend	Python 3.11.x, FastAPI	NLP microservice
Backend	Maven	Build and dependency management
NLP & Machine Learning	spaCy (primary), NLTK (fallback)	Lemmatization and POS tagging
NLP & Machine Learning	sentence-transformers / SBERT, Hugging Face Transformers	Embeddings (code_search_net for sentences, BGE for keywords)
NLP & Machine Learning	PyTorch	Backend engine for ML models
AI Services	LLM API	Semantic validation in requirement analysis
AI Services	LLM	Semantic code analysis (chunking, feature extraction)
AI Services	OpenAI	AI-assisted code suggestions
Data & Infrastructure	MySQL	Shared relational database
Data & Infrastructure	REST APIs, API Gateway	Single entry point for all services
Data & Infrastructure	SonarQube	Standalone static code analysis
Frontend	React	Web interface
Tooling	Git	Version control and collaboration
Hardware	Standard development laptops	No specialized hardware required

Table 1: Tools and Technologies Used

1.7 Project Timeline / Gantt Chart

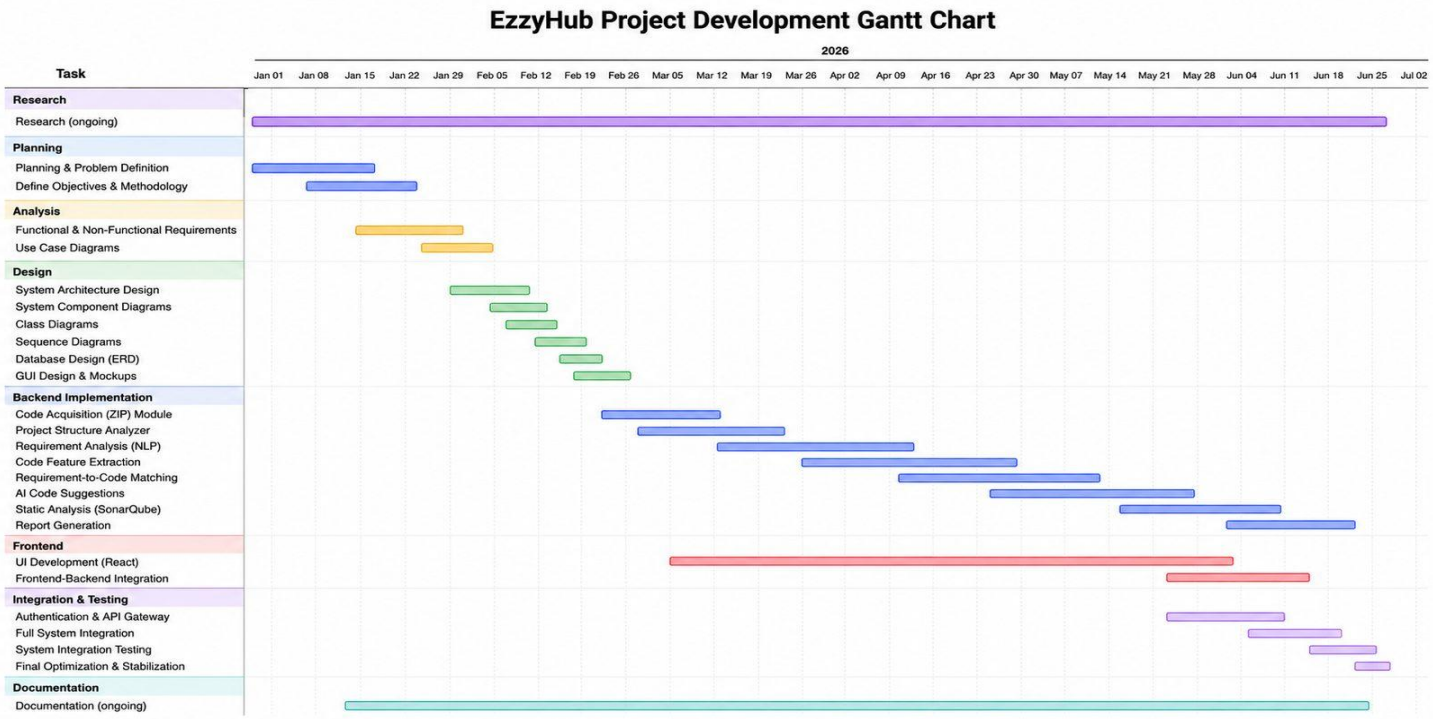


Figure 3: Use Case Diagram of the EzzyHub Platform

1.8 Report Organization

This report is organized as follows. Chapter 1 introduces the problem, objectives, scope, and methodology. Chapter 2 reviews related work and positions EzzyHub against existing tools. Chapter 3 presents the system analysis, including stakeholders and functional and non-functional requirements. Chapter 4 describes the system design (architecture, class and sequence diagrams, database, and security). Chapter 5 covers the implementation details. Chapter 6 presents testing and evaluation results. Chapter 7 concludes with a summary, lessons learned, and future work.

Chapter 2: Related Work

2.1 Overview

Several categories of tools address parts of the problem EzzyHub targets AI code generation, static code analysis, executability-focused generation, code feature extraction, and requirements traceability. Each contributes valuable capabilities, but none provides an integrated system that simultaneously verifies functional-requirement coverage and offers AI-assisted gap resolution. This section reviews representative tools and approaches and identifies the limitation each leaves open.

- 2.1.1 AI-Assisted Software Development Tools
- 2.1.2 Static Code Analysis (SonarQube-style tools)
- 2.1.3 ZenCoder (Generative Trial-and-Error)
- 2.1.4 Static Code Feature Extraction SootFX
- 2.1.5 Requirements Traceability Feature Extraction
- 2.1.6 AutoOAS

2.2 Comparative analysis: What distinguishes this project from existing work?

The tools reviewed above each address one dimension of the problem, but leave a gap that EzzyHub is designed to fill. The key distinction is that EzzyHub unifies **requirement awareness**, **semantic requirement-to-code matching**, **AI-assisted gap resolution**, and **standalone static analysis** in a single platform, rather than treating them separately.

Tool / Approach	What it does	Limitation	How EzzyHub differs
AI code tools (ChatGPT, Copilot, Claude)	Generate and complete code	No guarantee of requirement coverage; may omit imports/config	Verifies which requirements are actually implemented, and reports coverage
SonarQube-style static analysis	Detect bugs, vulnerabilities, code smells	No requirement awareness	Adds requirement-to-code matching on top of (separate) static analysis

ZenCoder	Iterative generate-test-fix for executability	Prioritizes execution, ignores requirement compliance	Measures requirement compliance, not just whether code runs
SootFX (feature extraction)	Extracts code-level features for ML	Code-level only; no semantic bridge to requirements	Bridges the semantic gap between NL requirements and code
Requirements traceability (IR-based)	Links requirements to code via IR heuristics	Heuristic; struggles with new patterns	Uses semantic embeddings (SBERT) plus keyword matching for more robust links
AutoOAS	Generates API docs from Spring Boot code	Documentation only; no requirement validation	Focuses on verifying requirement implementation, not documentation

Table 2: Comparison of EzzyHub with Existing Tools

In summary, existing tools are strong within their own scope but fragmented. EzzyHub's contribution is an **integrated verification pipeline**: it combines natural-language requirement analysis, semantic and keyword-based matching, coverage classification, and AI-assisted suggestions, while also offering standalone static analysis closing the gap that each individual tool leaves open.

2.3 All sources properly cited using IEEE format

[1] K. Karakaya and E. Bodden, "SootFX: A Static Code Feature Extraction Tool for Java and Android," in Proc. IEEE Int. Working Conf. Source Code Analysis and Manipulation (SCAM), 2021. (SootFX 2.1.4)

[2] Y. Wang et al., "Feature extraction for improving IR-based requirements traceability recovery," (journal/conference, year fill in the exact venue for the Wang et al. study you cited). (Requirements traceability 2.1.5)

[3] (AutoOAS) add the citation for the AutoOAS paper you referenced. (2.1.6)

[4] (SonarQube) SonarSource, "SonarQube," [Online]. Available: https://www.sonarsource.com/products/sonarqube/ (2.1.2)

[5] (ZenCoder) add the source/reference you used for ZenCoder. (2.1.3)

[6] (AI tools) if you cite specific limitations of ChatGPT/Copilot/Claude, add the source (paper or report) those figures/claims came from. (2.1.1)

Chapter 3: System Analysis

3.1 Stakeholder Analysis

1. Primary Stakeholders

1.1 Software Developers: Verify code executability, ensure requirement coverage, and get AI-assisted improvement suggestions.
1.2 AI-Assisted Development Users: Ensure AI-generated code is correct, complete, and safe to integrate.
1.3 Project Teams: Maintain consistency between requirements and implementation, detect gaps, and reduce integration issues.

2. Secondary Stakeholders

2.1 Project Managers: Receive requirement coverage reports to track progress and reduce project risks.
2.2 QA Engineers: Identify executability issues and missing functionality early to reduce defects.
2.3 Software Architecture Designers: Analyze project structure and detect inconsistencies or missing components.

3. Academic and Educational Stakeholders

3.1 Students: Learn the link between requirements and implementation, identify mistakes, and practice verification.
3.2 Instructors / Teaching Assistants: Efficiently evaluate assignments with objective requirement matching and executability reports.

4. System Administrators

4.1 System Administrators: Maintain EzzyHub platform, manage users, ensure system availability, and monitor performance.

3.2 Functional Requirements

FR-1: User Account Management

FR-1.1 User Registration The system shall:

- FR-1.1.1 Allow a new user to create an account using username/email and password.
- FR-1.1.2 Validate user input (required fields, valid email format).
- FR-1.1.3 Prevent registration using an already existing email/username.
- FR-1.1.4 Securely store user credentials after successful registration.

FR-1.2 User Login The system shall:

- FR-1.2.1 Allow registered users to log in using valid credentials.
- FR-1.2.2 Deny access if credentials are incorrect.
- FR-1.2.3 Display an appropriate error message for invalid login attempts.

FR-2: Code Acquisition

FR-2.1 File Upload Support The system shall:

- FR-2.1.1 Allow users to upload a project as a ZIP file.
- FR-2.1.2 Accept ZIP files up to 50 MB.
- FR-2.1.3 Extract all source code files from the archive.
- FR-2.1.4 Validate ZIP integrity.
- FR-2.1.5 Handle nested directory structures.
- FR-2.1.6 Reject non-ZIP files with an error message.

FR-2.2 Project Structure Analysis The system shall:

- FR-2.2.1 Preserve the original package/folder hierarchy.
- FR-2.2.2 Identify source files and group them by package/module.
- FR-2.2.3 Detect the main application entry point where applicable.

- FR-2.2.4 Identify configuration files.
- FR-2.2.5 Create an internal representation of the project structure.

FR-3: Requirements Processing

FR-3.1 Natural-Language Requirement Parsing The system shall:

- FR-3.1.1 Accept 1–100 functional requirements.
- FR-3.1.2 Split text into individual requirements.
- FR-3.1.3 Handle multi-line requirements.
- FR-3.1.4 Ignore empty lines and headers.

FR-3.2 Keyword Extraction The system shall:

- FR-3.2.1 Remove stop words.
- FR-3.2.2 Identify meaningful terms.
- FR-3.2.3 Extract nouns, verbs, and action verbs.
- FR-3.2.4 Convert words to their base form (lemmatization).
- FR-3.2.5 Produce semantic embeddings for each requirement.

FR-3.3 Requirement Validation The system shall:

- FR-3.3.1 Check minimum requirement length (≥ 5 words).
- FR-3.3.2 Warn if no action verbs are detected.
- FR-3.3.3 Perform AI semantic validation to confirm the text is a genuine, single, clear requirement (skippable per request).
- FR-3.3.4 Display validation warnings before processing.

FR-4: Code Structure Analysis

FR-4.1 Source Code Parsing The system shall:

- FR-4.1.1 Parse the uploaded source files.
- FR-4.1.2 Extract class/type names and their containing packages/modules.
- FR-4.1.3 Identify component types (e.g. Controller, Service) where applicable.
- FR-4.1.4 Extract method/function signatures.
- FR-4.1.5 Extract code features used for matching (via semantic code analysis).
- FR-4.1.6 Handle parsing errors gracefully.

FR-5: Requirement-to-Code Matching

FR-5.1 Similarity Matching The system shall:

- FR-5.1.1 Compare requirement keywords and embeddings against code feature keywords and embeddings.
- FR-5.1.2 Calculate a similarity score using combined semantic (cosine) and keyword (Jaccard) similarity.
- FR-5.1.3 Normalize keywords and consider synonyms.
- FR-5.1.4 Generate a similarity score between 0.0 and 1.0 for each requirement–feature pair.

FR-5.2 Automatic Status Classification The system shall:

- FR-5.2.1 Classify each requirement using thresholds: score ≥ 0.80 → IMPLEMENTED; 0.40–0.79 → PARTIAL; < 0.40 → MISSING.
- FR-5.2.2 Store both the score and the status for each requirement.

FR-5.3 Coverage Calculation The system shall:

- FR-5.3.1 Count requirements by status (Implemented, Partial, Missing).
- FR-5.3.2 Calculate coverage percentage: (Implemented / Total) × 100.
- FR-5.3.3 Display coverage and a breakdown by status in the report.

FR-6: AI-Assisted Suggestions

FR-6.1 Suggestions for Missing & Partial Requirements The system shall:

- FR-6.1.1 Identify requirements with status PARTIAL or MISSING.
- FR-6.1.2 Send the requirement (with project context and technology stack) to the AI suggestion service (OpenAI).
- FR-6.1.3 Generate candidate source code with a brief explanation.
- FR-6.1.4 Provide the generated code as a separate code report.

FR-7: Static Analysis (SonarQube)

Standalone stage independent of the matching and AI-suggestion flow.

FR-7.1 Static Code Analysis The system shall:

- FR-7.1.1 Perform static analysis on the uploaded project using SonarQube.
- FR-7.1.2 Detect bugs, security vulnerabilities, and code smells.
- FR-7.1.3 Detect execution-preventing issues such as missing imports.
- FR-7.1.4 Produce a separate static-analysis report of the findings.

FR-8: Report Management

FR-8.1 Report Generation The system shall generate reports combining the analysis results. The system produces three separate reports:

1. **Matching report** coverage percentage, per-requirement status (color-coded), match scores, and matched code features.
2. **Code report** the source files generated by the AI suggestion service for partial/missing requirements.
3. **Static-analysis report** bugs, vulnerabilities, and code smells from SonarQube.

FR-9: Logout

- FR-9.1 Allow user logout.
- FR-9.2 Terminate the session securely.

3.3 Non-Functional Requirements

NFR-1: Performance

- NFR-1.1 Verification processing time: small projects (1–10 files) ≤ 30 s; medium (11–50 files) ≤ 2 min; large (51–100 files) ≤ 4 min.
- NFR-1.2 Up to 100 requirements shall be parsed within 8 seconds.
- NFR-1.3 Source files shall be analyzed at a rate of 2–3 files per second on standard hardware.

NFR-2: Scalability

- NFR-2.1 The system shall handle projects containing up to 100 source files with up to 5,000 lines per file.
- NFR-2.2 The system shall process up to 100 functional requirements without failure.
- NFR-2.3 The system shall not exceed 4 GB RAM during analysis of a single project.
- NFR-2.4 The system shall support a growing number of users without noticeable performance degradation.

NFR-3: Reliability

- NFR-3.1 Validated changes shall not introduce unexpected failures in 95% of cases.
- NFR-3.2 The system shall maintain at least 90% uptime during testing and demonstrations.
- NFR-3.3 The system shall detect and report processing failures gracefully rather than crashing.
- NFR-3.4 The system shall maintain 95% availability for login and registration, and authentication data shall not be lost on failure.

NFR-4: Usability

- NFR-4.1 The system shall provide a simple main interface with a limited number of primary inputs.
- NFR-4.2 The system shall provide immediate visual feedback when a request is initiated.
- NFR-4.3 The system shall display clear, non-technical error messages.
- NFR-4.4 Reports shall be easy to read and use visual indicators to highlight results.

NFR-5: Security

- NFR-5.1 All user inputs shall be validated to prevent SQL injection, XSS, and path traversal attacks.
- NFR-5.2 Passwords shall be encrypted during storage and transmission, and accounts shall be limited after multiple failed login attempts.

3.4 Use Case

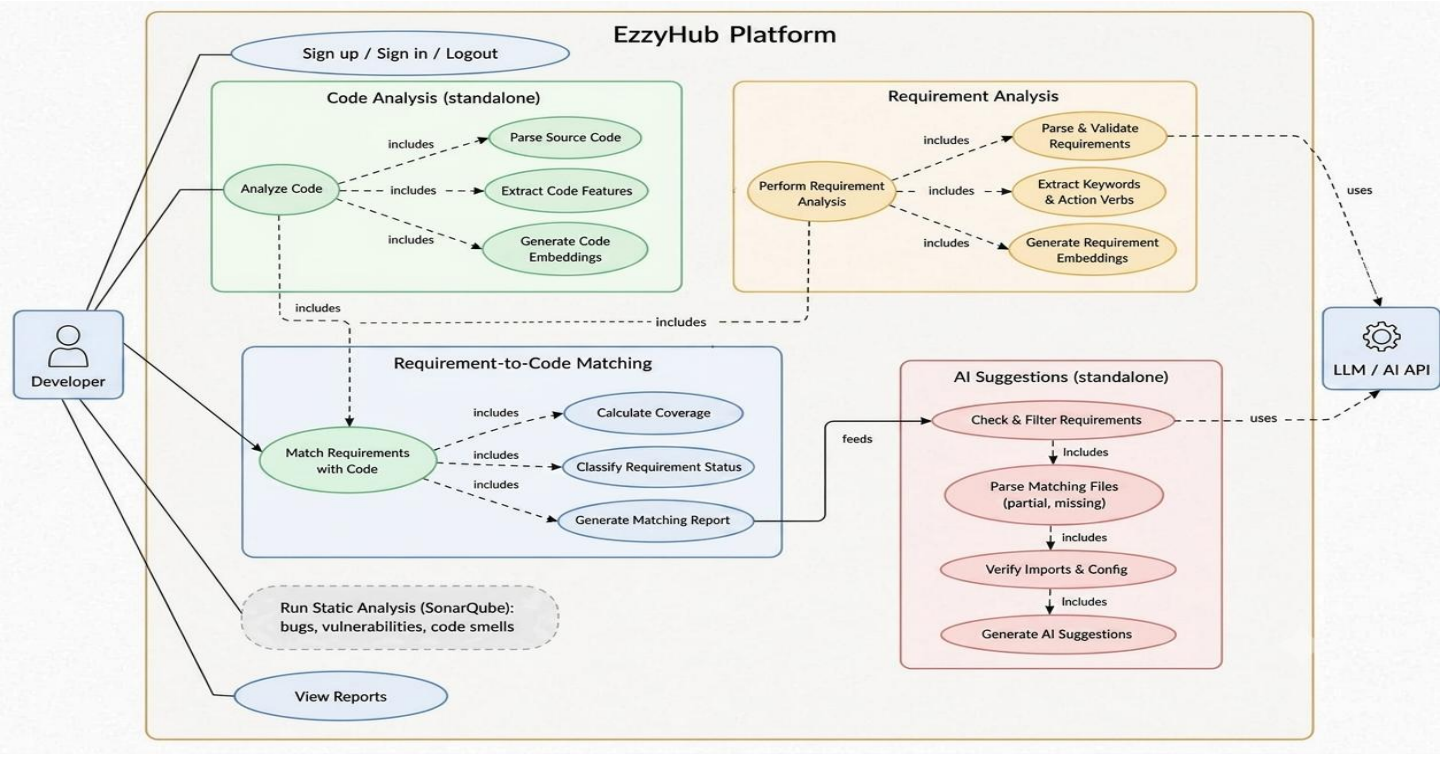


Figure 4: EzzyHub System Architecture

Actor Descriptions

Developer (Primary Actor)

The Developer is the main user of the EzzyHub platform and initiates all core operations. The Developer can:

- Register, log in, and log out of the platform.
- Upload a project (as a ZIP file) and enter functional requirements.
- Run **Code Analysis** independently to analyze a project's structure and extract its code features and embeddings.
- Run **Requirement-to-Code Matching**, which internally performs requirement analysis and code analysis, calculates coverage, classifies each requirement's status, and generates the matching report.
- Request **AI Suggestions** to generate candidate code for partial or missing requirements.
- Run **Static Analysis (SonarQube)** independently to detect bugs, vulnerabilities, and code smells.
- View the generated reports.

LLM / AI API (Secondary / External Actor)

The LLM / AI API is an external system that EzzyHub relies on to perform intelligent language tasks. It is invoked by the platform (not directly by the Developer) in three places:

- During **requirement analysis**, to semantically validate that a submitted requirement is genuine, single, and clear.
- During **code analysis**, to semantically analyze the code and extract its features.
- During **AI suggestions**, to generate candidate code for unimplemented requirements.

The LLM is a supporting actor: it responds to the platform's requests and returns results, but it does not initiate any use case on its own.

3.5 Feasibility Study (technical, economic, and operational brief)

Technical Feasibility: EzzyHub is technically feasible given its reliance on mature, well-supported technologies Spring Boot for backend services, FastAPI for NLP processing, MySQL for persistence, and React for the frontend. The microservice architecture, though complex to orchestrate locally, allows each component (requirement analysis, code analysis, matching, AI suggestions) to be developed and scaled independently. External LLM providers (Groq, Cohere, OpenAI) supply the AI capabilities without requiring in-house model training, reducing technical risk. The main technical challenges lie in coordinating nine services with shared authentication and a common database schema, which the team addressed through consistent environment-variable-based configuration and a shared JWT secret.

Economic Feasibility: Development costs are primarily limited to developer time, since the platform relies on open-source frameworks (Spring Boot, FastAPI, React) with no licensing fees. The main recurring costs are API usage fees for the three external LLM providers and, optionally, hosting for a SonarQube instance and cloud infrastructure if deployed beyond local development. Given these are pay-per-use services, costs scale with actual usage, making the system economically viable for both small-scale testing and gradual production rollout without significant upfront investment.

Operational Feasibility: The system is operationally feasible for its intended users developers and teams verifying that code matches software requirements since it automates a process that would otherwise require manual review. The startup procedure, while involving multiple services, is documented and reproducible via a fixed sequence of commands. Health-check endpoints on every service simplify monitoring and troubleshooting. The main operational consideration is that the JDK version split (17 vs. 21) and multiple environment variables require careful setup, which the documented startup order and configuration conventions are designed to mitigate.

Chapter 4: System Design

4.1 System Architecture

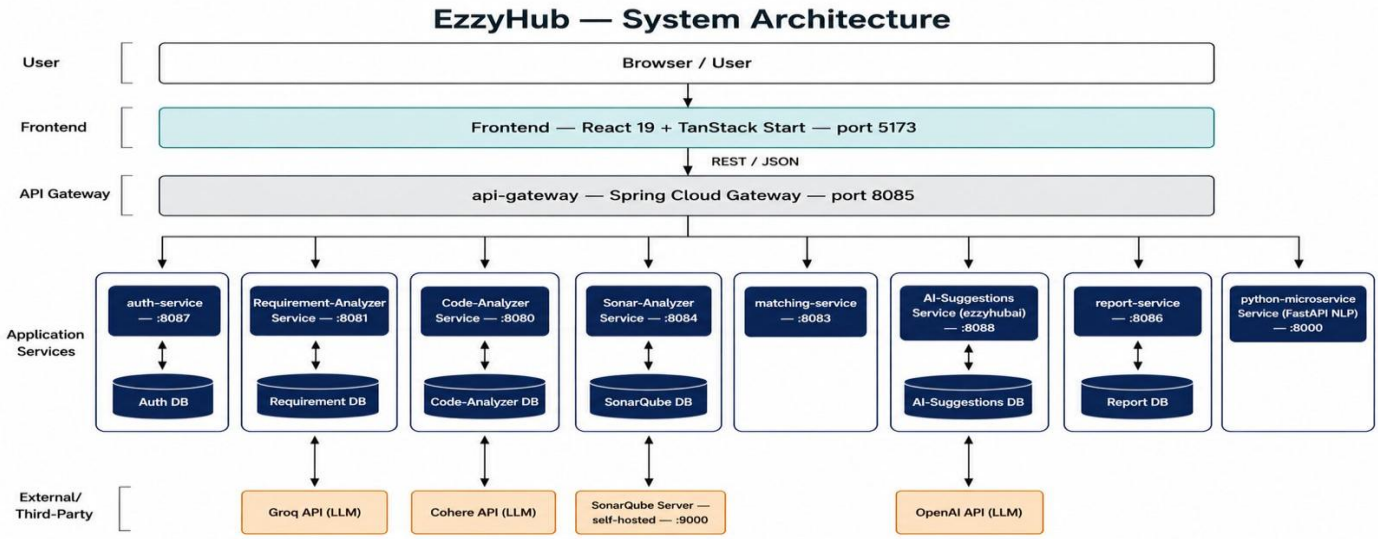


Figure 5: System Architecture Diagram

Component Diagram

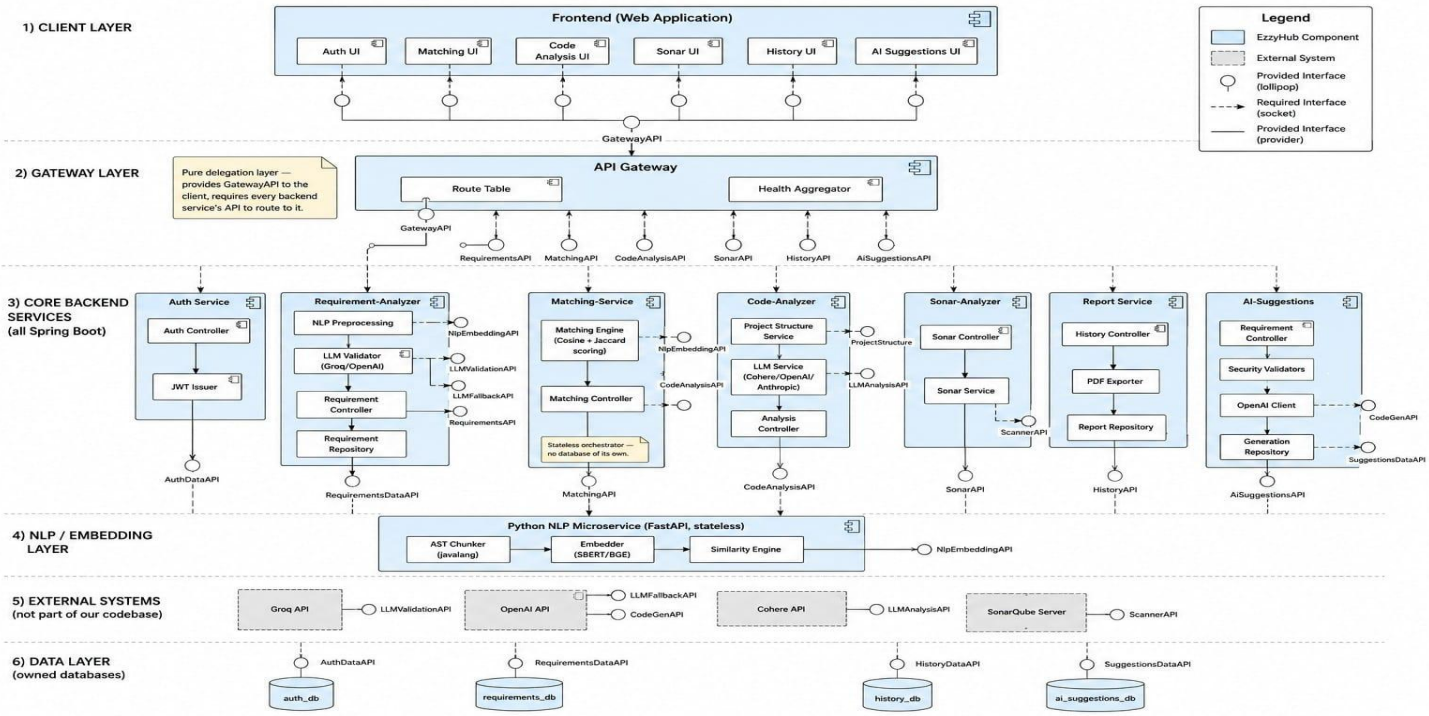


Figure 6: System Component Diagram

4.2 Class Diagrams

4.2.1 Requirement Analysis

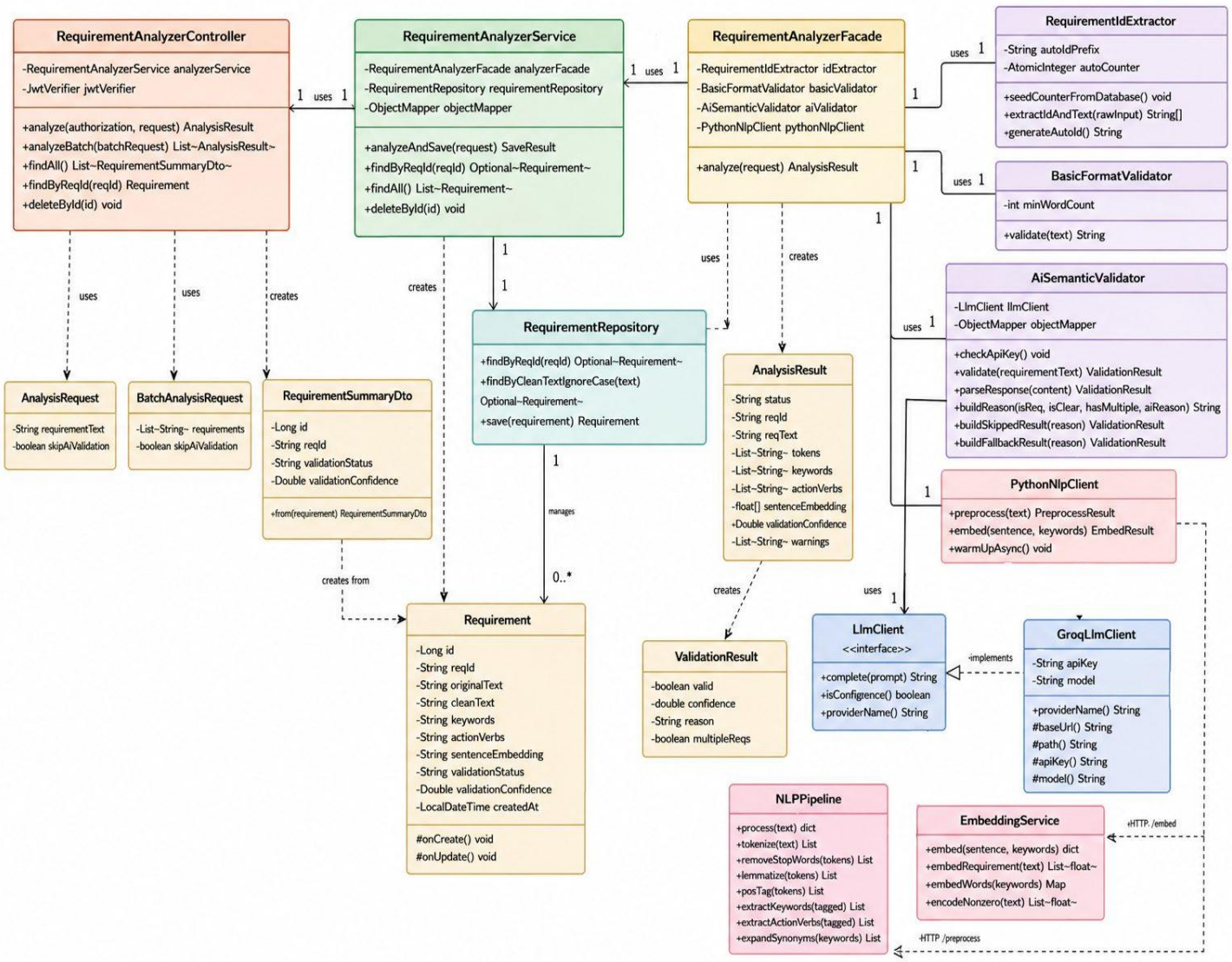


Figure 7: Class Diagram Requirement Analysis

4.2.2 Static Analysis by sonar

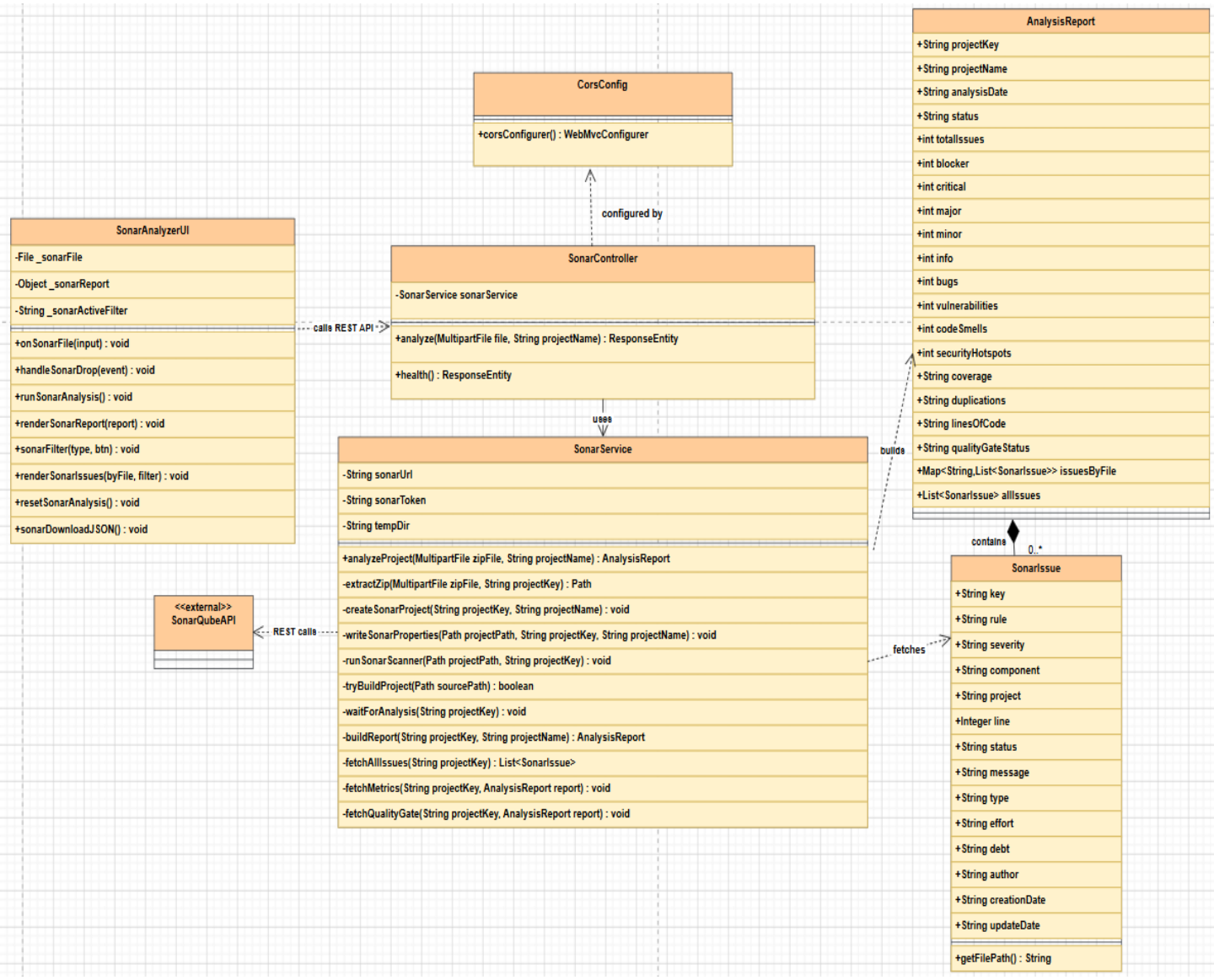


Figure 8: Class Diagram Static Analysis (Sonar)

4.2.3 Matching Service

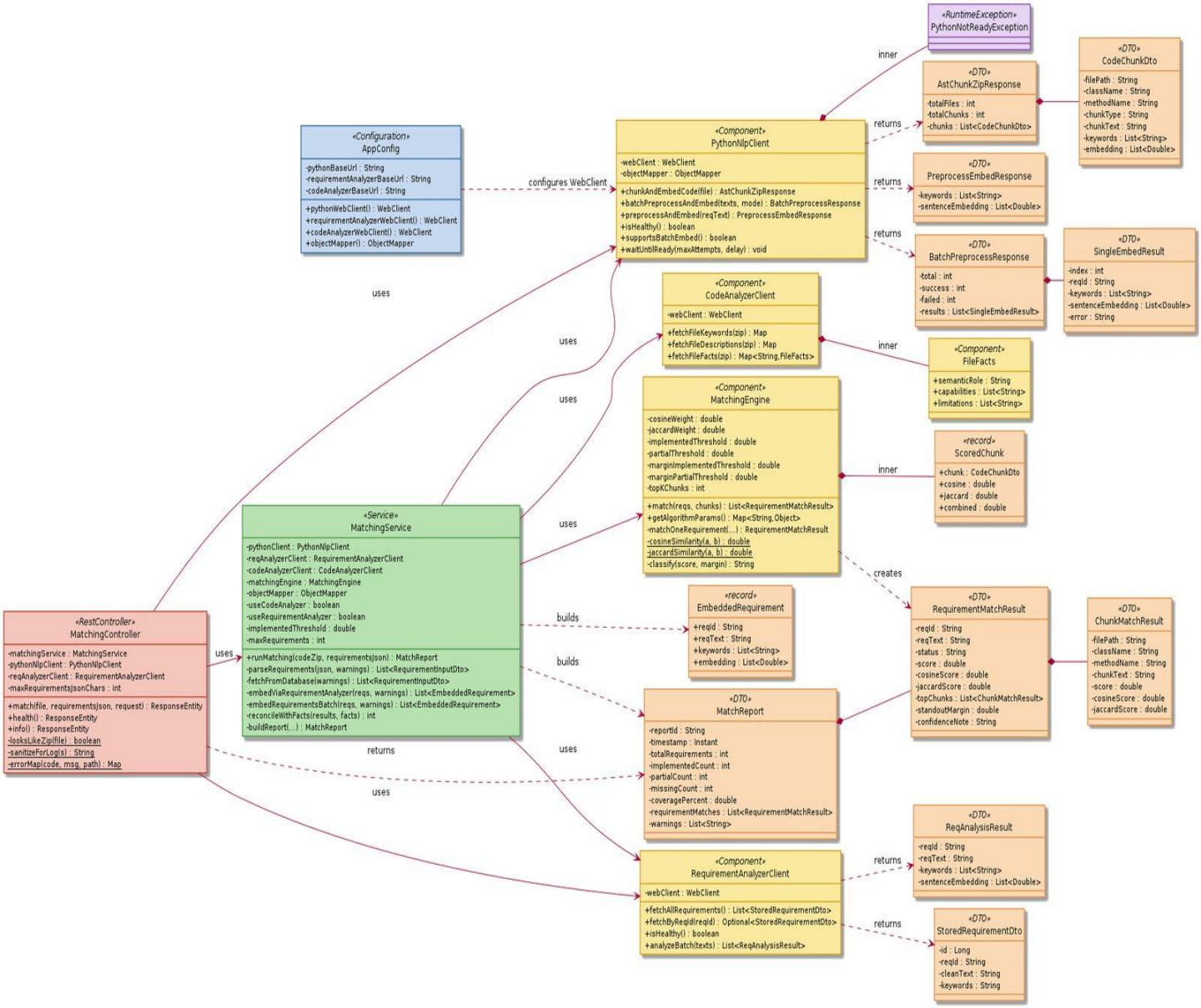


Figure 9: Class Diagram Static Analysis (Sonar)

4.2.4 Code Analyzer

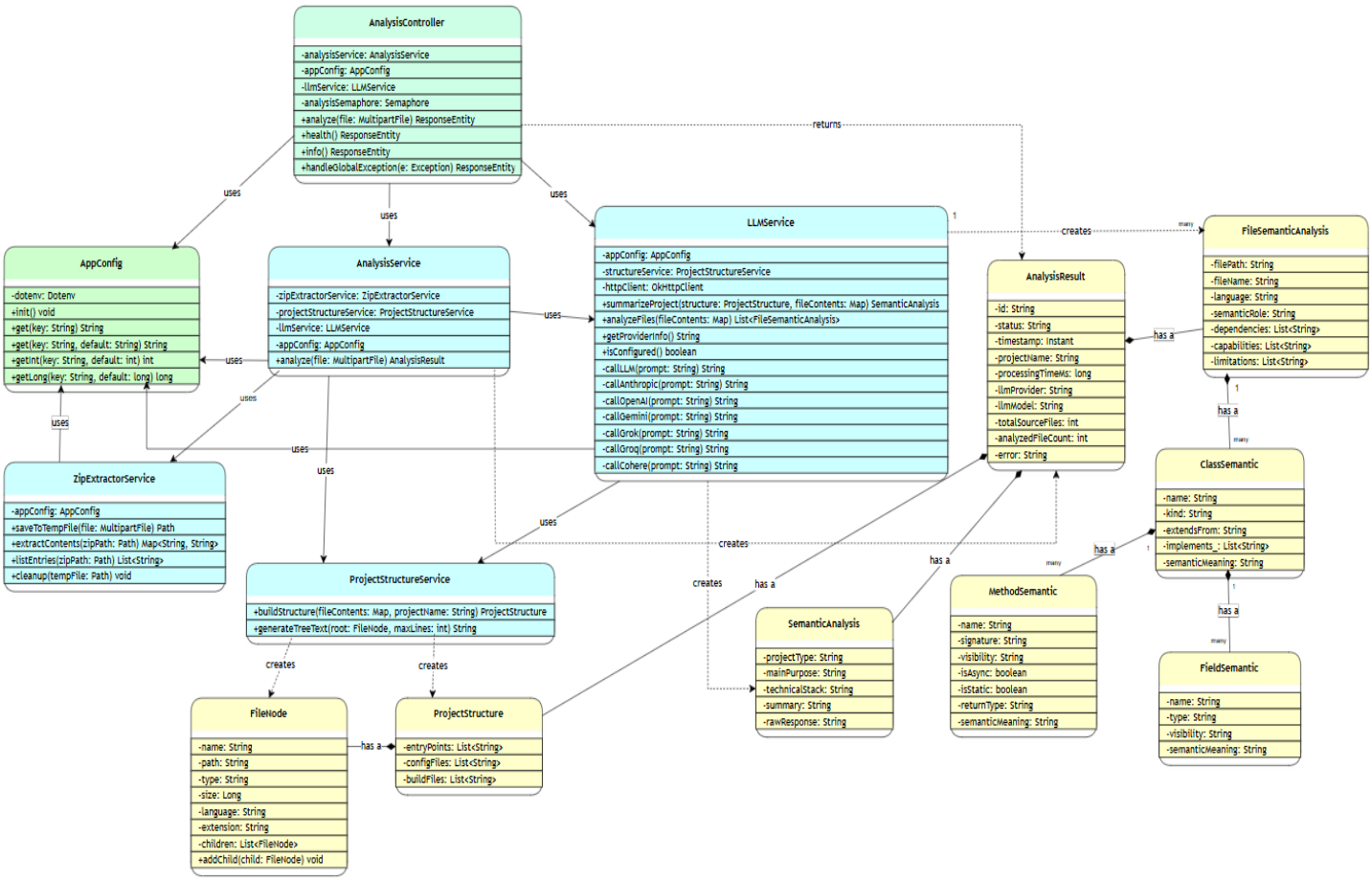


Figure 10: Class Diagram Code Analyzer

4.2.5 AI Suggestions

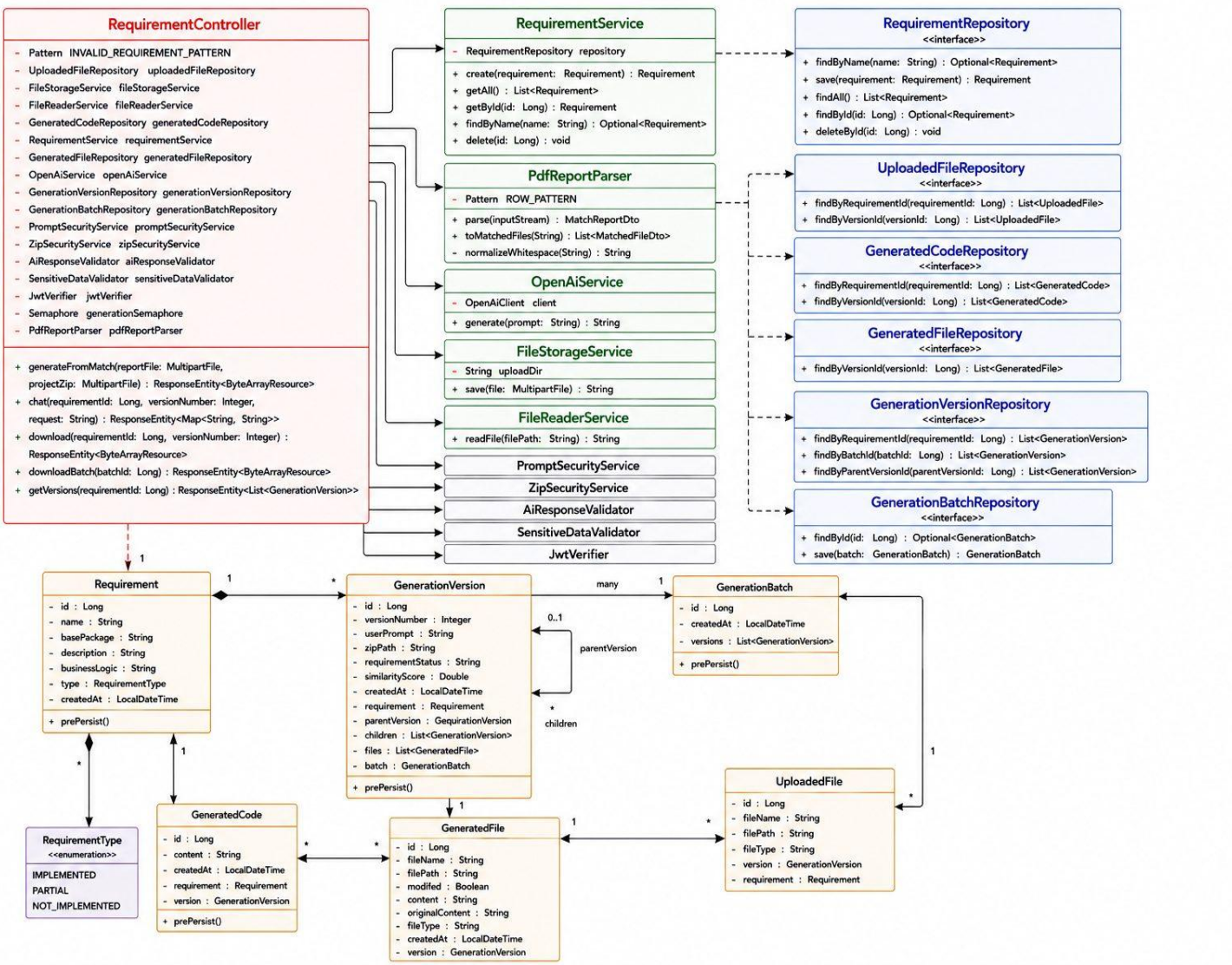


Figure 11: Class Diagram AI Suggestions

4.3 Sequence Diagrams (for key use cases)

4.3.1 Requirement Analysis

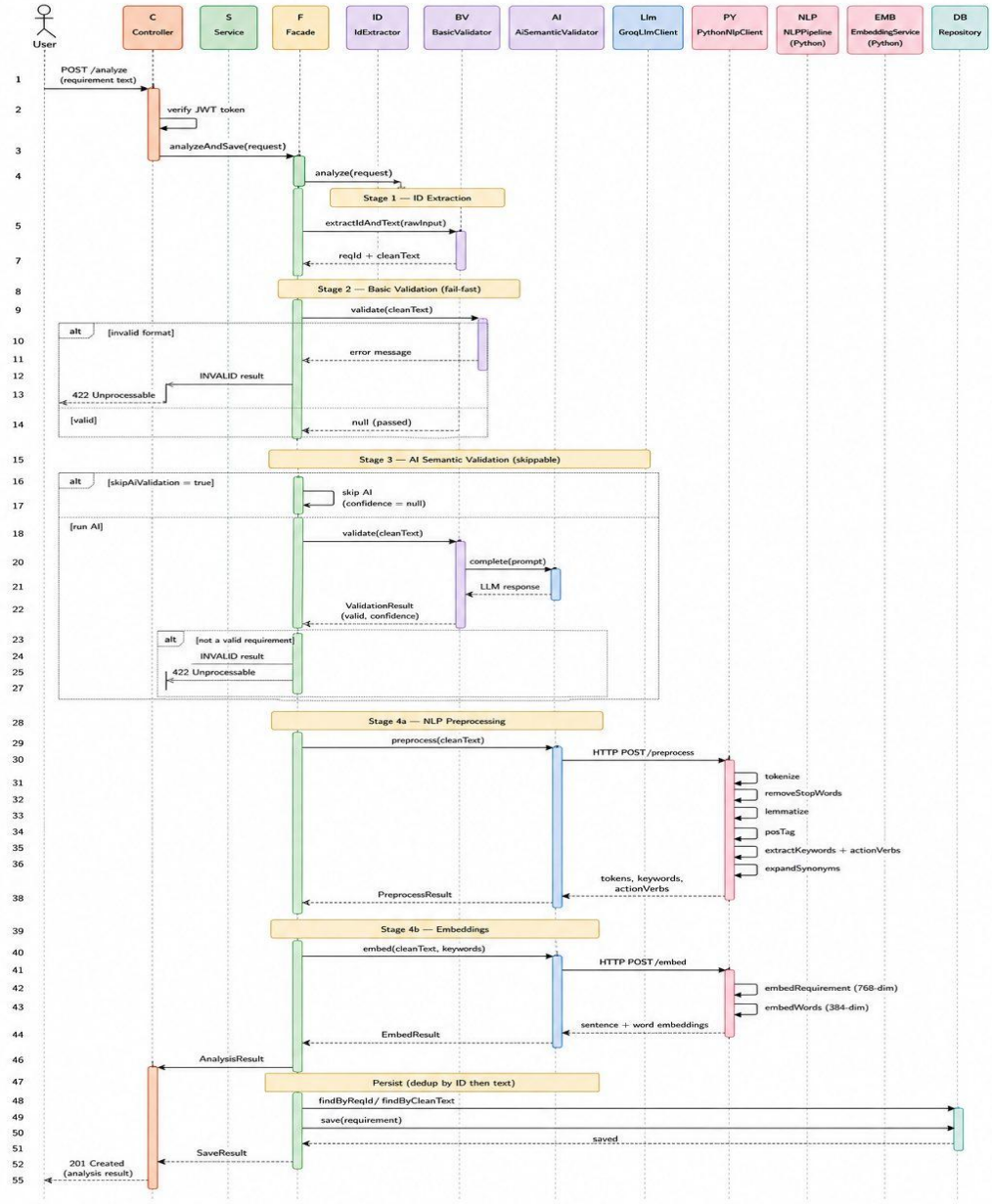


Figure 12: Sequence Diagram Requirement Analysis

4.3.2 Static Analysis by Sonar

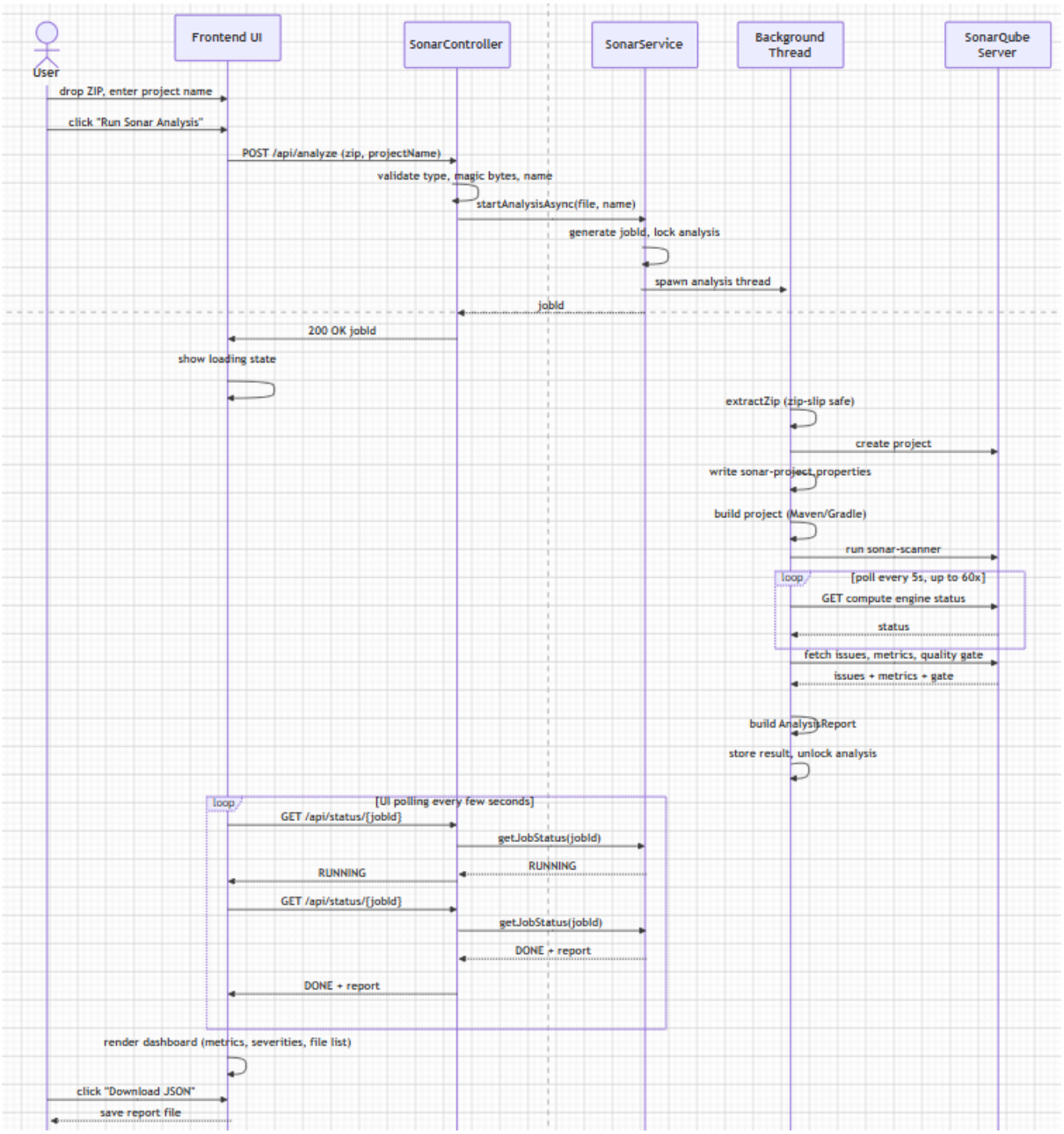


Figure 133: Sequence Diagram Static Analysis (Sonar)

4.3.3 Matching Service

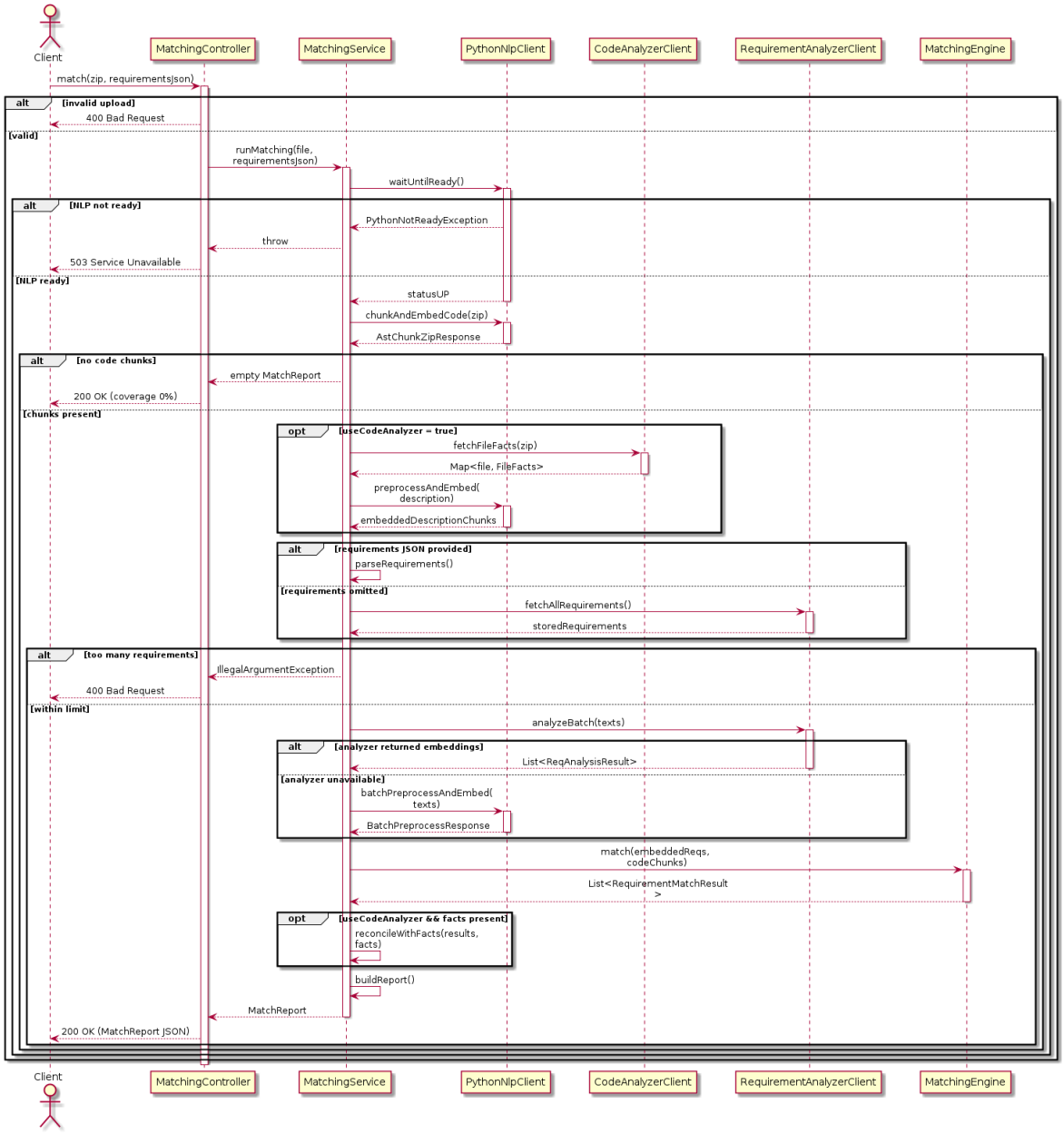


Figure 24: Sequence Diagram Matching Service

4.3.4 Code Analyzer

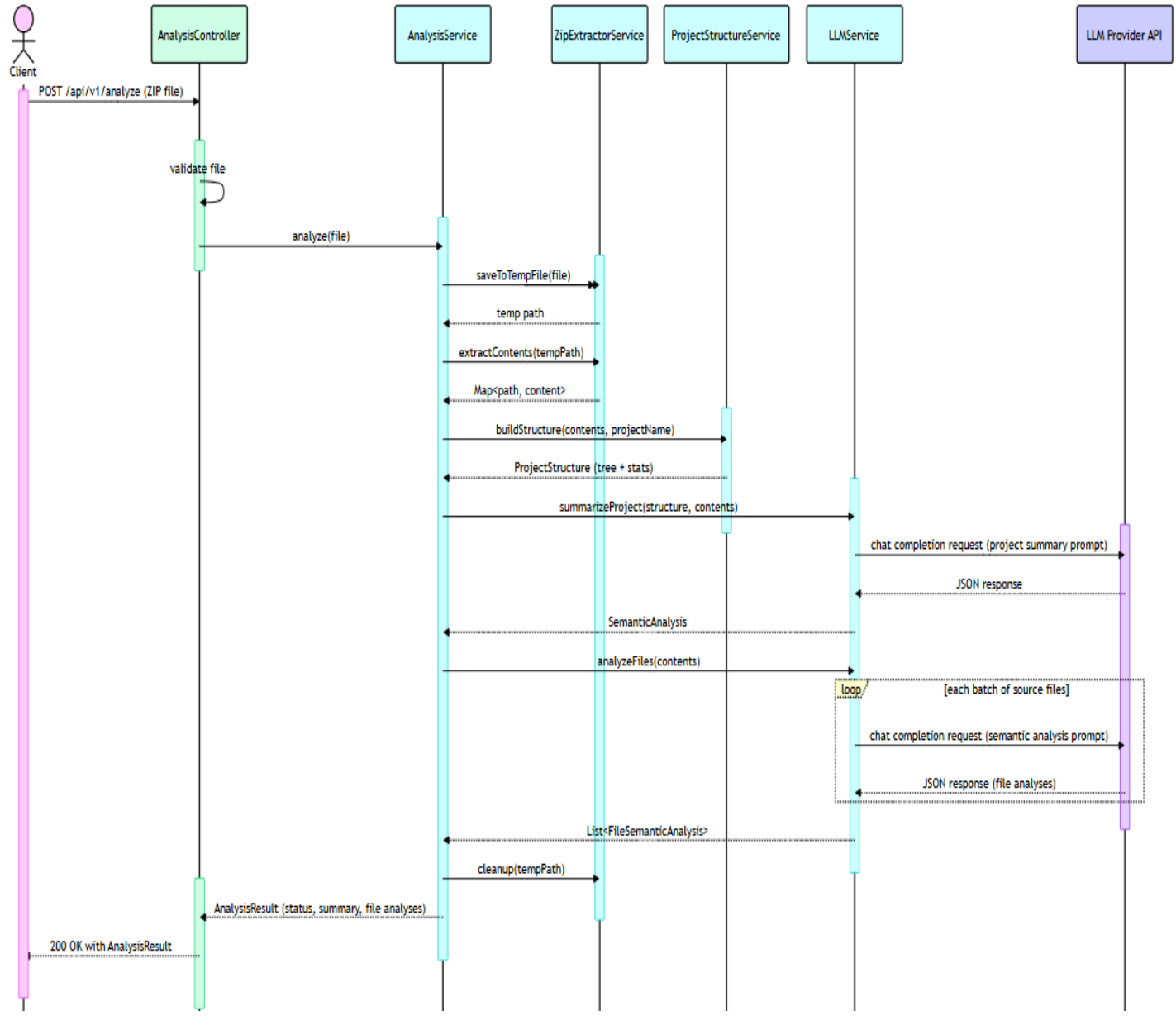


Figure 25: Sequence Diagram Code Analyzer

4.3.5 AI Suggestions

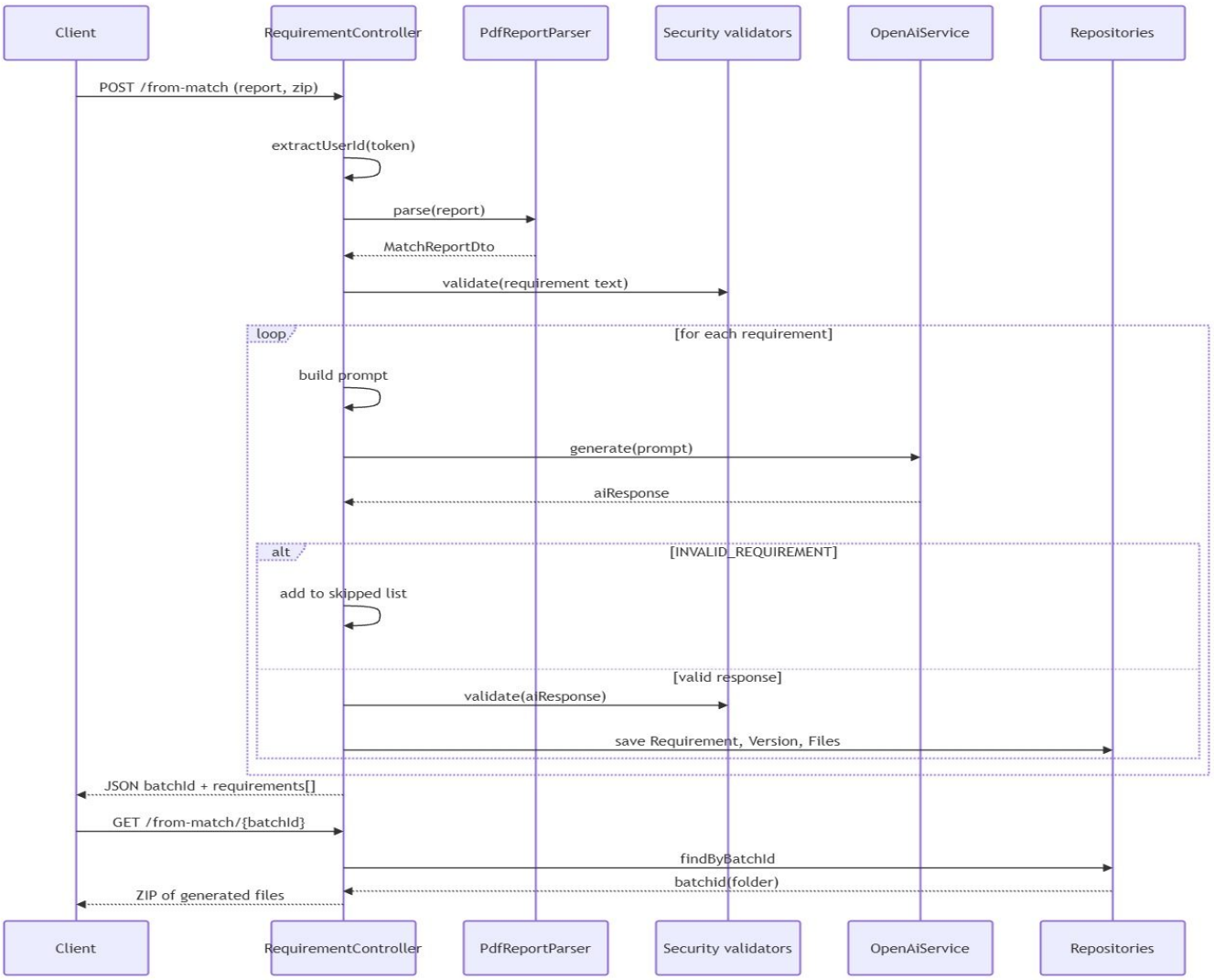


Figure 14: Sequence Diagram AI Suggestions

4.4 Database Entity-Relationship Diagram (ERD)

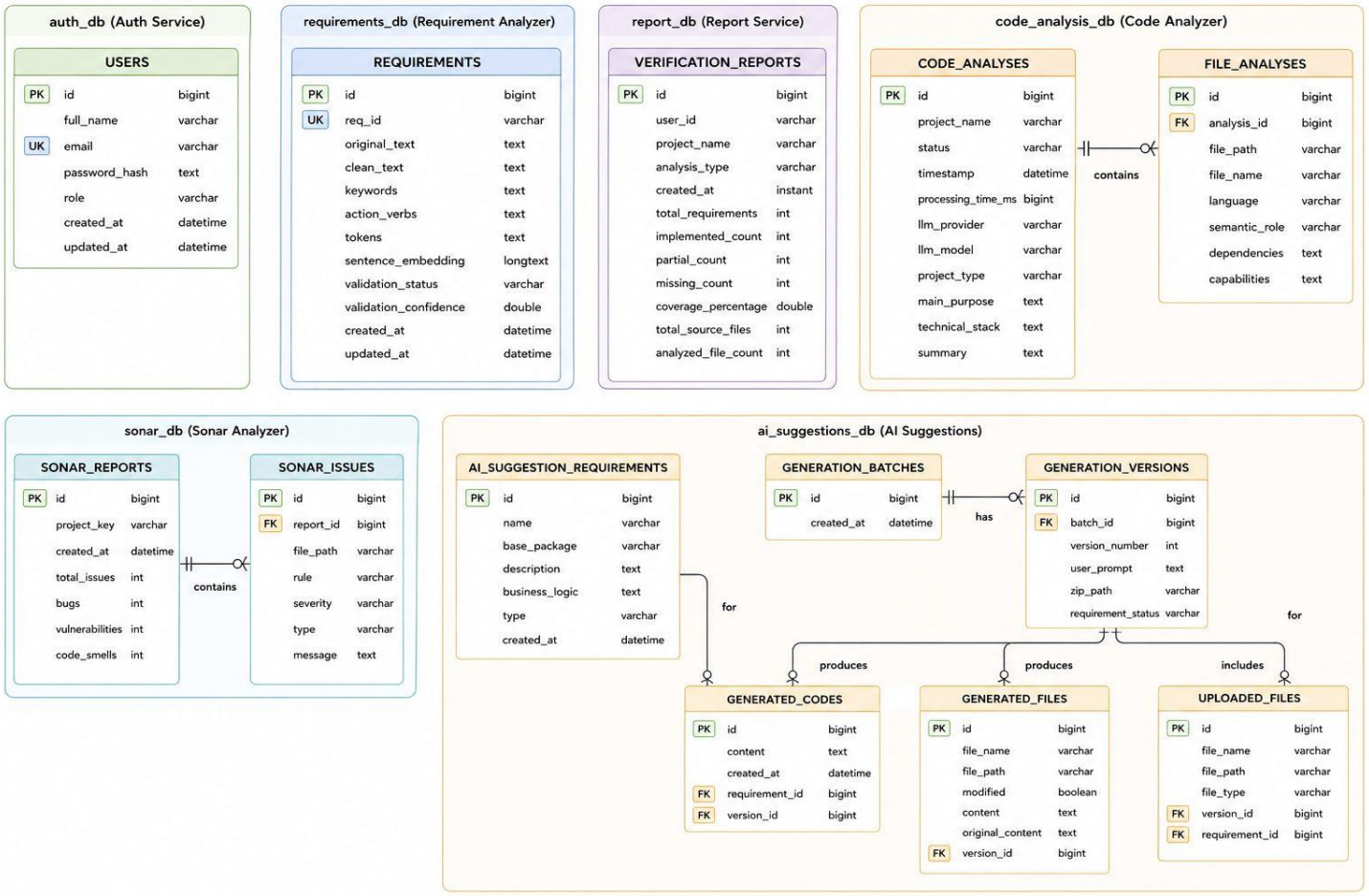
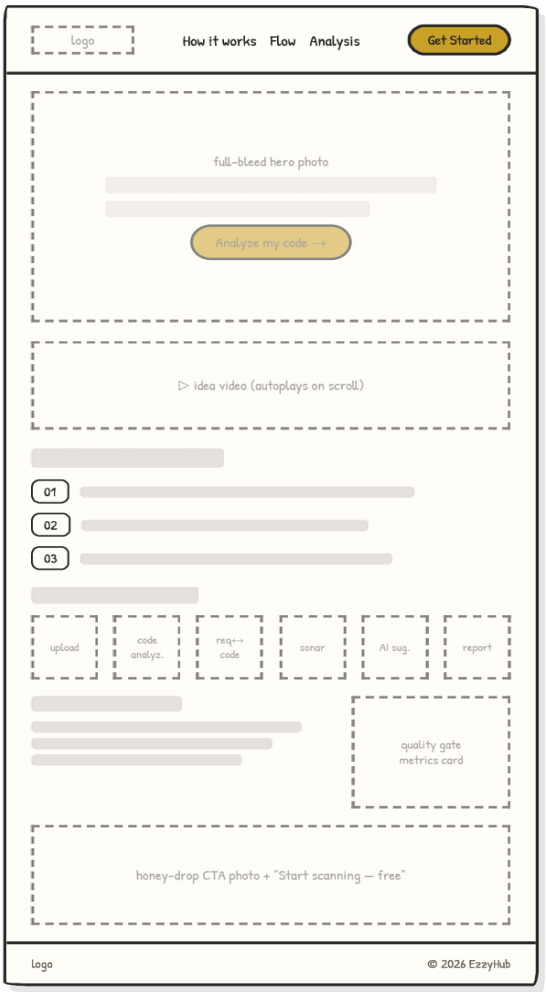


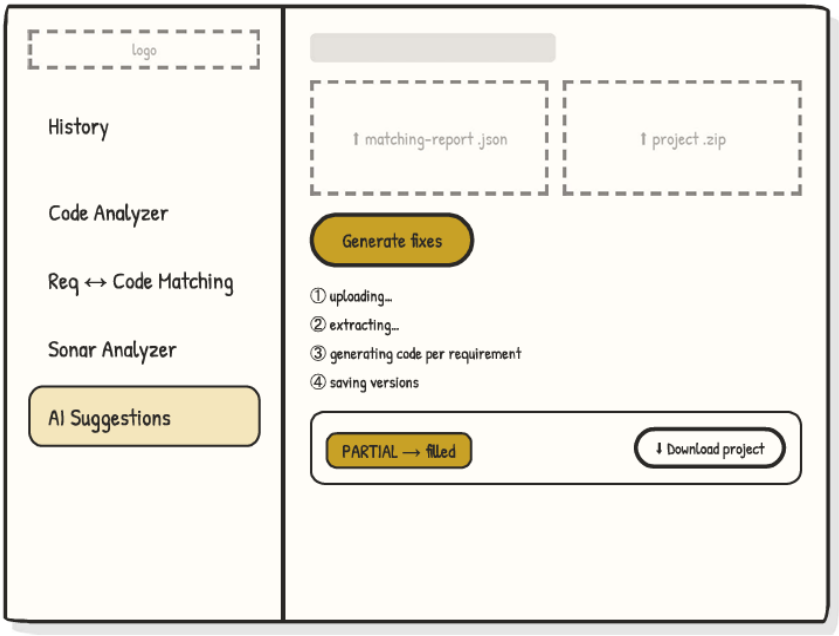
Figure 15: Database Entity-Relationship Diagram (ERD)

4.5 User Interface Wireframes / Mockups

1a Landing page



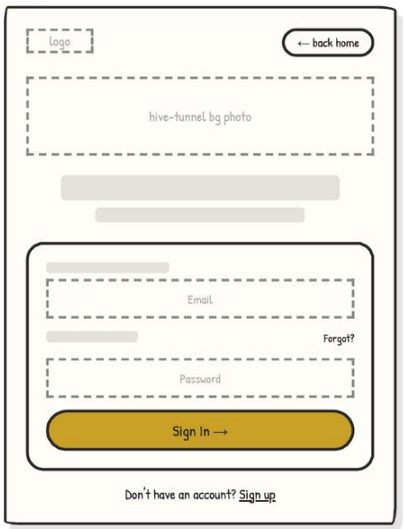
1h Dashboard — AI Suggestions



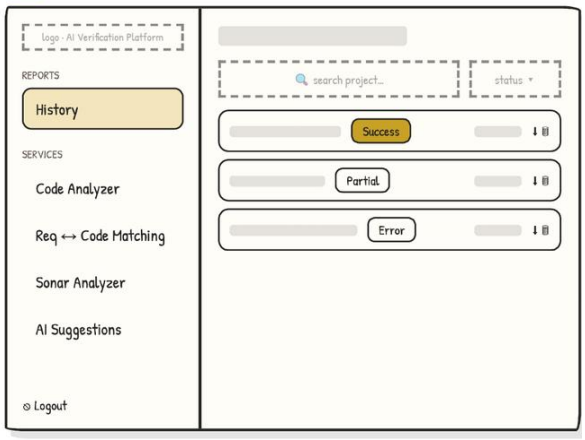
1b Sign up



1c Sign in



1d Dashboard — History (default view)



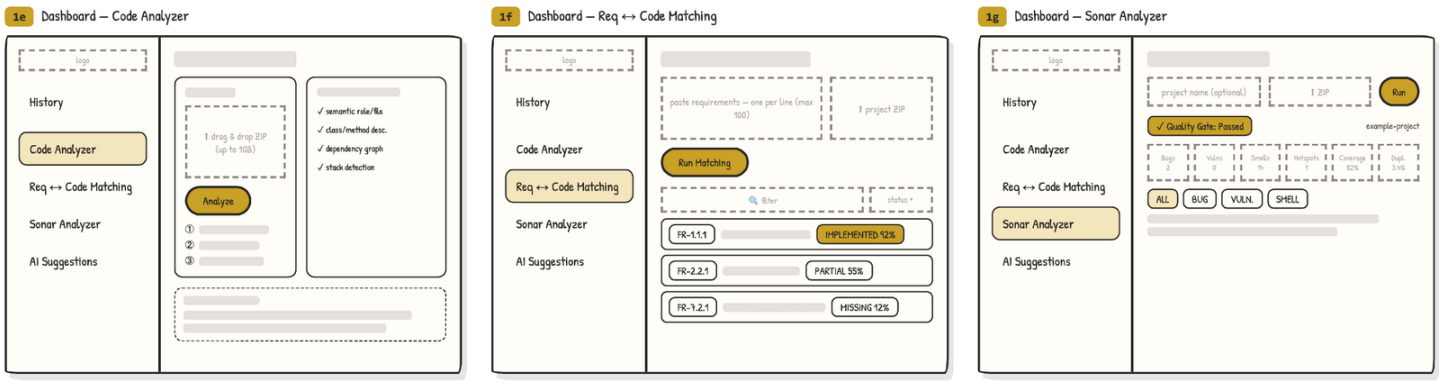


Figure 16: User Interface Wireframes

4.6 Security Design Considerations

4.6.1 Requirement Analysis

Authentication and authorization: User-facing endpoints are protected by JWT verification: each request must carry a valid bearer token, which is checked before any processing. All services that issue or verify tokens share a single secret provided through an environment variable (AUTH_JWT_SECRET), never hard coded.

Input validation and safe failure: Incoming payloads are validated with Bean Validation (e.g. non-blank text, length bounds), and the Python microservice independently validates request schemas with Pydantic (bounded text length, keyword-count caps). Requirement content is additionally checked by the basic and AI validators before processing.

Error handling without information leakage: A centralized @RestControllerAdvice handler maps each failure type to the correct HTTP status (401 unauthorized, 422 invalid requirement, 400 malformed input, 500 internal) and returns a uniform, non-technical error body. Full exception detail is logged server-side only; internal messages, stack traces, and database cause messages are never exposed to the client.

Secrets and configuration: API keys and database credentials are externalized to environment variables rather than committed to source control. The Python microservice refuses to start in non-

development mode unless a service key is supplied (fail-loud), preventing an unprotected service from running unnoticed.

Data-layer safety: Persistence uses parameterized JPA/Hibernate queries, which prevent SQL injection, and database writes run inside transactions, so a mid-operation failure roll back cleanly rather than leaving partial data.

4.6.2 Static Analysis by Sonar

Zip-slip protection: every extracted entry's resolved path is validated against the target directory before being written, rejecting archives that attempt path traversal (e.g. "../../").

File-type verification: uploads are validated both by extension and by inspecting the file's magic bytes, so a renamed non-ZIP file is rejected even if it passes the extension check.

Input whitelisting: project names and job IDs are validated against strict regular expressions, preventing injection through user-supplied identifiers that are later used in file paths and process arguments.

No hardcoded secrets: the SonarQube host URL and authentication token are injected through externalized configuration (environment variables), removing credentials from source control.

Controlled process execution: build and scan commands are invoked through a fixed, predefined argument list rather than a shell string, avoiding command injection.

Restricted CORS policy: cross-origin access is scoped to the API path with an explicit method whitelist rather than a fully unrestricted configuration.

Concurrency lock: a single in-progress analysis is enforced at a time, reducing the risk of resource-exhaustion from parallel, uncontrolled uploads.

4.6.3 Matching Service

File-type verification: uploads are validated both by extension and by inspecting the file's magic bytes (PK signature), so a renamed non-ZIP file is rejected with HTTP 400 before it is buffered or forwarded to downstream services.

Bounded resource consumption: the inline requirements payload is capped both in size (1,000,000 characters, checked cheaply in the controller) and in count (2,000 requirements, enforced before the embedding stage). Because each requirement triggers at least one downstream embedding call, these caps bound the fan-out a single request can amplify into (OWASP LLM "Unbounded Consumption"). Abusive payloads are rejected with a clear 400; normal usage is unaffected.

Error handling without information leakage: on internal failure the client receives only a generic message plus a generated correlation identifier, while the full exception detail (internal service names, ports, downstream response bodies) is logged server-side under that identifier. Controlled responses (503 when the NLP service is not ready, 400 validation messages) are preserved unchanged.

Log-injection prevention and sensitive-data hygiene: control characters are stripped from every user-controlled value (e.g. uploaded filenames) before it reaches logs or error bodies, preventing forged log entries (CWE-117). The default log level is INFO so requirement text and code snippets are never persisted to logs; DEBUG is reserved for a trusted development profile.

Safe CORS configuration: credentials are disabled (the API is credential-less) and the allowed origins are externalized to a configuration property, so production can pin them to the real frontend host without a code change. This removed a wildcard-origin-with-credentials combination that effectively disabled the browser's same-origin protection.

Vetted serialization instead of hand-rolled code: request bodies sent to the Python NLP service are serialized with Jackson rather than manual string concatenation, guaranteeing that all JSON control characters in user-controlled requirement text are escaped correctly (no JSON-injection breakout).

No SSRF or deserialization surface: all outbound service URLs are fixed configuration, never derived from user input, and JSON is parsed without polymorphic/default typing into concrete DTOs.

Patched dependencies: the framework was upgraded from Spring Boot 3.2.3 to the final 3.2.x patch release (3.2.12), removing published CVEs in Spring Web, Tomcat, and Netty (URL-parsing flaws, static-resource path traversal, content-negotiation DoS) with a behavior-preserving, patch-only change.

Fail-closed readiness gate: if the Python NLP service is down or still loading its models, the request is rejected with an explicit 503 rather than silently producing a report built from zero-vector embeddings (which would present false results as valid output).

4.6.4 Code Analyzer

Authentication and authorization: The service does not perform its own token verification; it sits behind the API Gateway and is not intended to be reached directly, so identity checks are handled at the perimeter rather than duplicated here.

Input validation and safe failure: Uploaded archives are validated before processing file type, non-empty content, size, and entry-count limits are enforced, and archive entries attempting path traversal are rejected rather than extracted.

Error handling without information leakage: A centralized exception handler maps each failure to the correct HTTP status (400 invalid input, 429 over capacity, 500 internal) with a uniform, non-technical message. Full exception detail, including any third-party provider errors, is logged server-side only and never returned to the client.

Secrets and configuration: LLM provider API keys are externalized to environment variables rather than committed to source control.

Data-layer safety: The service is stateless and holds no persistent data store, so it carries no database-layer attack surface of its own.

4.6.5 AI Suggestions

Because the AI-Suggestions service accepts untrusted user input (requirement text, chat messages,

uploaded ZIP archives) and forwards parts of it to an external LLM, several security concerns specific to AI-integrated systems were addressed at the design level, in addition to standard authentication controls.

Authentication via shared JWT verification. The service does not issue its own tokens. Instead, JwtVerifier validates JWTs issued by the separate auth-service, using a shared HMAC signing secret (auth.jwt.secret). This keeps token issuance centralized in one service while allowing AI-Suggestions to independently verify identity on every protected endpoint via the Authorization: Bearer <token> header. Missing headers, malformed tokens, invalid signatures, and expired tokens are all rejected as UnauthorizedException, mapped to a 401 response. The secret is defensively padded to 32 bytes at startup so a short override value cannot crash the service, mirroring the same safeguard used in the organization's other services (report-service, Code-Analyzer) that reuse this secret.

Prompt injection defense. Because user-supplied text (requirement descriptions, chat messages, and report contents) is embedded directly into prompts sent to the LLM, PromptSecurityService screens all such text against a list of known injection patterns phrases attempting to override system instructions, extract the system prompt, or invoke dangerous system-level actions before it is included in any prompt. This check runs on the raw user input and again on the fully assembled prompt, since a benign-looking requirement combined with matched file content could still form an injection payload once concatenated.

Sensitive data leakage prevention. SensitiveDataValidator inspects both outbound prompts and inbound AI responses for patterns resembling API keys, database credentials, JWT secrets, and connection strings (e.g. sk-, jdbcpostgresal://, jwt.secret). This guards against two distinct risks: accidentally forwarding secrets from a project's real files into the LLM prompt, and the LLM echoing back sensitive-looking strings (whether real or hallucinated) in generated code that later gets persisted and returned to the client.

AI response contract validation. Because the service relies on the LLM to follow a strict output format (=== FILE === / === CODE === / === END === , or one of the fixed status markers),

AiResponseValidator rejects any response that is empty or does not conform to this contract before it is parsed further. This prevents malformed or unexpected model output from silently producing corrupted GeneratedFile records.

ZIP archive hardening. Uploaded project archives are a common vector for zip-based attacks, so ZipSecurityService enforces several checks before and during extraction:

- A hard cap on the number of entries (zip.max-entries, default 20,000), rejecting the archive early if exceeded a defense against zip-bomb-style archives with excessive entry counts.
- Per-entry path validation rejecting ... , absolute paths (/, \), and drive-letter/colon patterns, preventing path traversal (zip-slip) attacks that could write files outside the intended extraction directory.
- The same path validation is reapplied to file paths the AI itself proposes for generated output, since a compromised or malformed model response could otherwise attempt to write outside the intended project structure.

Centralized, information-minimizing error handling. GlobalExceptionHandler ensures all exceptions are converted into a consistent JSON error shape, but deliberately limits what is disclosed to the client. Prompt-injection attempts, for instance, are logged server-side at WARN level with the specific detection detail, while the client only receives a generic "disallowed content" message preventing an attacker from using error responses to probe or tune around the detection rules. Unhandled exceptions are similarly logged in full server-side but surfaced to the client only as a generic 500 message, avoiding leakage of stack traces or internal implementation details.

Bounded concurrency for cost and availability control. The /from-match endpoint acquires a Semaphore before issuing OpenAI calls, capped at a configurable maximum (ai.max-concurrent-generations, default 3). Requests beyond this limit receive an immediate 429 Too Many Requests rather than queuing indefinitely, protecting against uncontrolled API cost and latency from bursts of concurrent generation requests.

Chapter 5: Implementation

5.1 Implementation Environment and Setup

Environment:

EzzyHub runs on a polyglot stack combining Java 17 (Java 21 for AI-Suggestions with Spring Boot 4.1.0) with Spring Boot 3.2.x for the other seven backend services, Python 3.11 with FastAPI for the NLP microservice, and Node.js 18+ with React 19 and TanStack Start for the frontend. MySQL 8.x provides storage . Three external LLM providers are integrated: Groq, Cohere, and OpenAI, each tied to a specific service. Maven 3.9+ manages Java builds, while npm handles frontend dependencies. An optional SonarQube instance supports static code analysis.

Setup:

Each of the nine backend services runs independently via Maven (mvn spring-boot:run) or Uvicorn, configured through environment variables including database credentials, API keys, and a shared AUTH_JWT_SECRET used by four services for consistent token verification. Services are started in dependency order python-microservice and Requirement-Analyzer first, since matching-service depends on both, followed by the remaining services, with api-gateway launched last since it aggregates health checks across all backend services. Ports were deliberately assigned to avoid conflicts, such as relocating Sonar-Analyzer from 8080 to 8084. The frontend is installed and run via npm install and npm run dev, communicating exclusively through the API Gateway rather than calling services directly. Setup concludes with verifying each service's individual health endpoint, followed by the gateway's aggregate check confirming the full platform is operational.

5.2 Key Implementation Details (algorithms, modules, major decisions)

5.2.1 Requirement Analyzer

The Requirement Analyzer transforms a natural-language functional requirement into a structured, comparable representation (ID, cleaned tokens, keywords, action verbs, and dense embeddings). It is implemented as a Java Spring Boot service that orchestrates the workflow and delegates the heavy

NLP and embedding work to a dedicated Python (FastAPI) microservice. This polyglot split lets each side use its strongest tools: Java for orchestration, validation, persistence, and REST, and Python for its mature NLP and machine-learning ecosystem (spaCy **[3]**, sentence-transformers **[1]**).

Four-stage analysis pipeline (Facade pattern)

The RequirementAnalyzerFacade coordinates the full pipeline behind a single analyze() entry point, hiding the internal complexity from callers. The stages run in a deliberate cheap-to-expensive order so invalid input is rejected before any costly work is done (fail-fast):

ID extraction RequirementIdExtractor recognises five ID formats (e.g. REQ-USER-1, RE-5, 1., [10]) via ordered regular expressions, and generates a sequential REQ-AUTO-N identifier when none is present.

Basic format validation BasicFormatValidator performs fast, cheap checks (minimum word count, presence of alphabetic content, at least three meaningful words). Failure short-circuits the pipeline immediately.

AI semantic validation AiSemanticValidator prompts a Large Language Model to confirm the text is a genuine, clear, single requirement.

NLP + embeddings the service calls the Python microservice for linguistic preprocessing and for sentence/word embeddings.

Natural-language processing (Python microservice)

The NLP pipeline (spaCy **[3]** primary, NLTK **[4]** fallback) executes tokenization, lowercasing, stop-word removal (standard English plus a curated requirements-domain stop list such as "shall", "system", "software"), lemmatization, part-of-speech tagging, and keyword and action-verb extraction. Two domain-specific decisions improve accuracy: action verbs are detected on the full sentence (preserving grammatical context), and synonym expansion uses a hand-curated software-domain map rather than WordNet **[5]**, which was disabled after it introduced semantic noise (e.g. expanding "user" to "drug user").

Semantic embeddings

Sentence embeddings **[1]** use the code_search_net model **[2]** (768-dimensional), which was trained specifically on natural-language-query to code-snippet pairs matching the project's task of aligning requirements with source code. This replaced a generic sentence model (all-MiniLM) and, on measured samples, widened the separation between relevant and irrelevant matches by roughly 72%, directly improving matching precision. Per-keyword embeddings use the BGE model **[6]** (384-dimensional). Both the requirement side and the code side share the same model and vector space, which is what makes their cosine similarity meaningful.

Major design decisions and resilience

Meaningful confidence in semantics. When AI validation is skipped by request, the confidence value is stored as null, so the report distinguishes "validation was not run" from a genuine borderline score.

Fail-open external validation. If the LLM is unreachable or unconfigured, the service treats the requirement as provisionally valid and continues, rather than blocking a sound requirement because of an external outage.

Resilient Java–Python integration. The PythonNlpClient raises the response buffer to 16 MB (long requirements produce large embedding payloads), warms the model at startup to avoid first-call timeouts, retries only transient failures (timeouts, connection errors, 5xx) with exponential backoff (up to 3 attempts, starting at 500 ms), and detects degenerate all-zero/NaN embedding vectors retrying once before falling back.

Deduplication. RequirementAnalyzerService avoids storing duplicates by matching on requirement ID first and then on cleaned text (case-insensitive), updating the existing row instead of creating a new auto-ID entry.

LLM settings. Requests use a low temperature (0.1) for deterministic validation output and a capped token limit (256) for predictable cost and latency.

5.2.2 Static Analysis by Sonar

Asynchronous Job-Based Pipeline

A SonarQube scan can take from seconds to minutes depending on project size, so the endpoint returns a **jobId** immediately and runs the analysis on a background thread. The client polls **GET /api/status/{jobId}** until the status changes to DONE or FAILED, avoiding long-held HTTP connections and request timeouts.

Analysis Pipeline

Each analysis follows a fixed sequence: (1) generate a unique project key, (2) safely extract the uploaded ZIP, (3) create the project in SonarQube via its API, (4) generate a **sonar-project.properties** file, (5) attempt to build the project (Maven or Gradle) so the scanner has compiled binaries for more accurate analysis, (6) run the Sonar Scanner as an external process, (7) poll the SonarQube Compute Engine until processing finishes, and (8) fetch issues, metrics, and quality-gate status to assemble the final report.

Report Aggregation

Results are aggregated into a single **AnalysisReport** model: issue counts by severity (blocker/critical/major/minor/info) and type (bug/vulnerability/code smell), coverage, duplication, lines of code, security hotspots, quality-gate status, and issues grouped by file giving the frontend everything needed to render both the summary and detailed views in one call.

Build-Tool Auto-Detection

The service detects whether the uploaded project uses Maven or Gradle and attempts a best-effort compile before scanning. If no build file is found, or the build fails, the analysis still proceeds using source-only analysis, ensuring the feature degrades gracefully rather than failing outright.

JSON Export

In addition to the on-screen report, the full analysis result can be exported as a downloadable JSON file useful for archiving results or generating a structured, AI-ready prompt for further review.

5.2.3 Matching Service

The Matching Service implements FR-7: for every functional requirement it decides whether the uploaded code IMPLEMENTS it, PARTIALLY implements it, or is MISSING it. It is a Java Spring Boot orchestrator that delegates AST parsing and embedding to the Python NLP microservice, optionally enriches results with the Code Analyzer's semantic output, and runs the core algorithm (the MatchingEngine) in Java.

AST-based chunking (Many-to-Many strategy)

A whole-file embedding mixes structure, imports, and several unrelated features, so it scores low against a short requirement even when the feature is implemented. The service therefore matches at the chunk level: the Python microservice parses each Java file into method/class chunks (javalang AST, regex fallback), embeds each chunk separately (384-dimensional SBERT), and the engine computes a Many-to-Many similarity matrix. Requirements and chunks share the same model and vector space, which is what makes their cosine similarity meaningful.

Matching pipeline

Each request follows a fixed sequence: (1) a readiness gate probes the Python service's health and aborts with HTTP 503 if it is not ready a cold server would otherwise produce zero-vector embeddings and false MISSINGs; (2) the ZIP is AST-chunked and embedded; (3) optionally, Code-Analyzer per-file descriptions are embedded as extra "description chunks" so requirements match against what the code does (best-effort, never breaks matching); (4) requirements come from the request body or the Requirement Analyzer database and are embedded through a three-level fallback chain (Requirement Analyzer → Python batch → serial calls); (5) the MatchingEngine scores and classifies; (6) rule-based reconciliation adjusts misleading verdicts; (7) a single MatchReport is assembled with coverage, results, parameters, timings, and warnings.

Hybrid scoring (cosine + Jaccard)

Combined score = 0.65 × cosine(embeddings) + 0.35 × Jaccard(keyword sets). Cosine captures semantic similarity; Jaccard captures exact lexical overlap on synonym-expanded keywords (so "login" matches "authenticate") and rescues matches cosine misses on abbreviations or domain-specific names. Both fail gracefully (zero vectors and empty sets score 0.0), and all weights and

thresholds are configurable. Only the best-scoring chunk of each file competes for the top-K, so one many-method class cannot crowd the correct match from another file out of the results.

Margin-aware classification

A raw threshold cannot distinguish a genuine match from the "best of a noisy lot" when shared vocabulary raises the whole score floor. Classification therefore also requires the top match to stand out: the top1–top2 score gap, normalized by the requirement's own score standard deviation. The margin thresholds (2.0 IMPLEMENTED, 1.0 PARTIAL) were calibrated by simulation at the real population size (226 candidate files): 500 pure-noise trials produced zero false IMPLEMENTED and ~96% fewer false PARTIALs. Each result carries a plain-language confidence note explaining its status.

Rule-based verdict reconciliation

Using Code-Analyzer facts (no extra LLM call), two rules downgrade misleading matches: a demanded quality (secure/validated/unique) that the matched file's limitations negate e.g. "store passwords securely" matched to code noting "passwords stored in plain text" and a demanded action verb (create, delete, logout, …) found in no file's capabilities. Downgrades are capped at one severity level per rule, and the rule is skipped when the fact pool is too small to be reliable.

Major design decisions and resilience

Every enrichment path degrades gracefully with an explicit report warning stating what quality was lost; only the readiness gate fails closed, since an explicit 503 beats a plausible-looking report built from zero vectors. The Requirement-Analyzer timeout was raised from 30 s to 300 s with retries, because cold-start model loading previously cut batches off and silently zeroed embeddings. Caller-supplied requirement IDs are never overridden; missing ones are auto-assigned (REQ-AUTO-N).

5.2.5 Code Analyzer

5.2 Key Implementation Details (Algorithms, Modules, Major Decisions)

The Code Analyzer service is implemented as a Spring Boot microservice with four cooperating components, coordinated by a single orchestration layer, with archive handling, structural analysis, and semantic (LLM-based) analysis kept as independent, separately-tested services.

Modules

Analysis Orchestration (AnalysisService). Coordinates the pipeline as four stages, save, extract, build structure, semantic analysis and tracks an overall outcome of success, partial, or error. If project-level summarization fails, a fallback summary is substituted and per-file analysis still proceeds, so one failed stage does not discard results already produced.

Archive Extraction (ZipExtractorService). Safely extracts untrusted ZIP uploads: enforces size and entry-count limits before processing, rejects path-traversal ("zip-slip") entries, and caps per-file read length independent of declared archive metadata. Temporary files are deleted once extraction completes, regardless of outcome.

Structural Analysis (ProjectStructureService). Builds a project tree and aggregate statistics (language/extension distribution, depth, entry points, build/config files) using static lookup tables keyed by filename and extension, rather than parsing file content, supporting many languages without a dedicated parser for each.

Semantic Analysis (LLMService). Builds prompts from project content, submits them to a configured LLM provider, and parses the response into structured semantic output. Implemented as a provider-agnostic adapter supporting six interchangeable LLM providers, selected via configuration.

REST Interface (AnalysisController). Limited to validation, concurrency-capacity enforcement, and delegation, no analysis logic itself, allowing independent testing via MockMvc.

Key Algorithms

- **Root-prefix normalization** strips a shared top-level folder from extracted paths so the tree is rooted at the project itself.

- **Importance-ranked file selection** scores source files (entry points and service/controller files ranked highest, tests lowest) and analyzes only the top-ranked subset, so limited LLM budget goes to the most architecturally significant files first.
- **Bounded batch processing** groups files into configurable batch sizes with a per-file character cap, keeping LLM cost and latency predictable regardless of project size.
- **Sensitive-file exclusion** filters out conventionally credential-bearing files (.env, private keys, service-account JSON, etc.) before any content reaches the LLM, regardless of importance ranking.
- **Tolerant output parsing** strips Markdown fences and extracts the JSON body from LLM responses; a parse failure is isolated to its batch, generating placeholder entries for just those files rather than failing the whole request.

Major Design Decisions

- **Graceful degradation:** a partial-success state reflects the decision that a structure tree and any completed semantic analysis are valuable even if another stage fails.
- **Provider-agnostic LLM integration:** avoids dependence on any single vendor's availability, pricing, or rate limits.
- **Stateless design:** no analysis data is persisted; that responsibility is delegated to the Report Service elsewhere in the system.
- **Filename/extension-based classification:** trades content-verified accuracy for broad language support without per-language parsers.

5.3 Sample Screenshots / Output Samples

5.3.1 Sign In

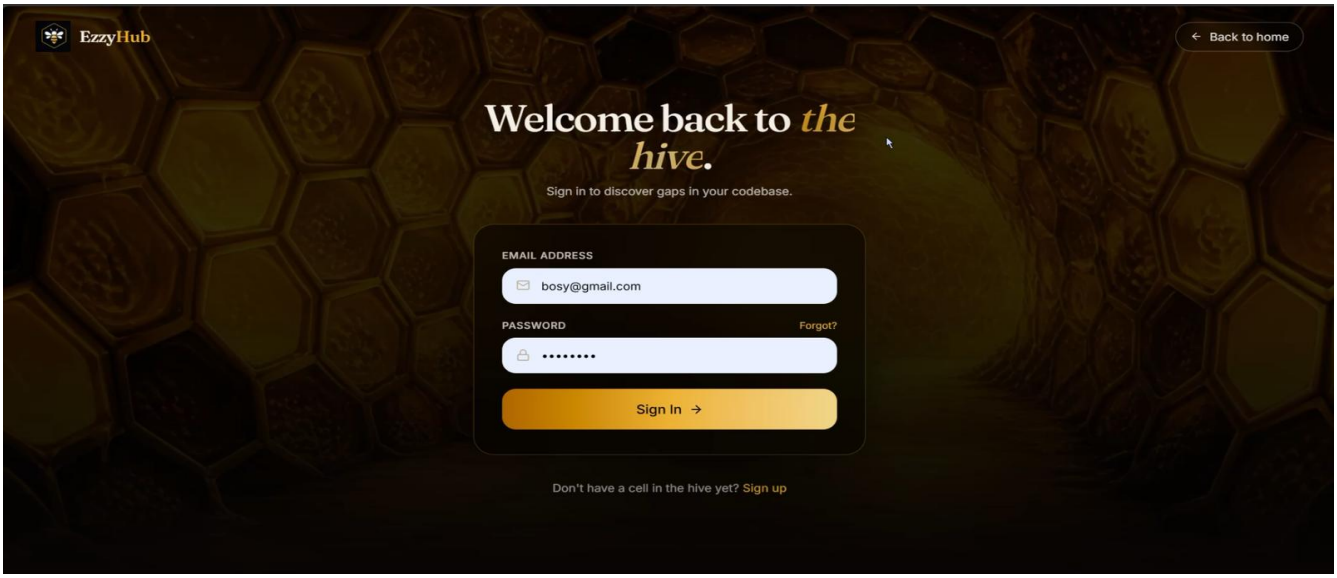


Figure 17: Sign In Screen

5.3.2 Sign Up

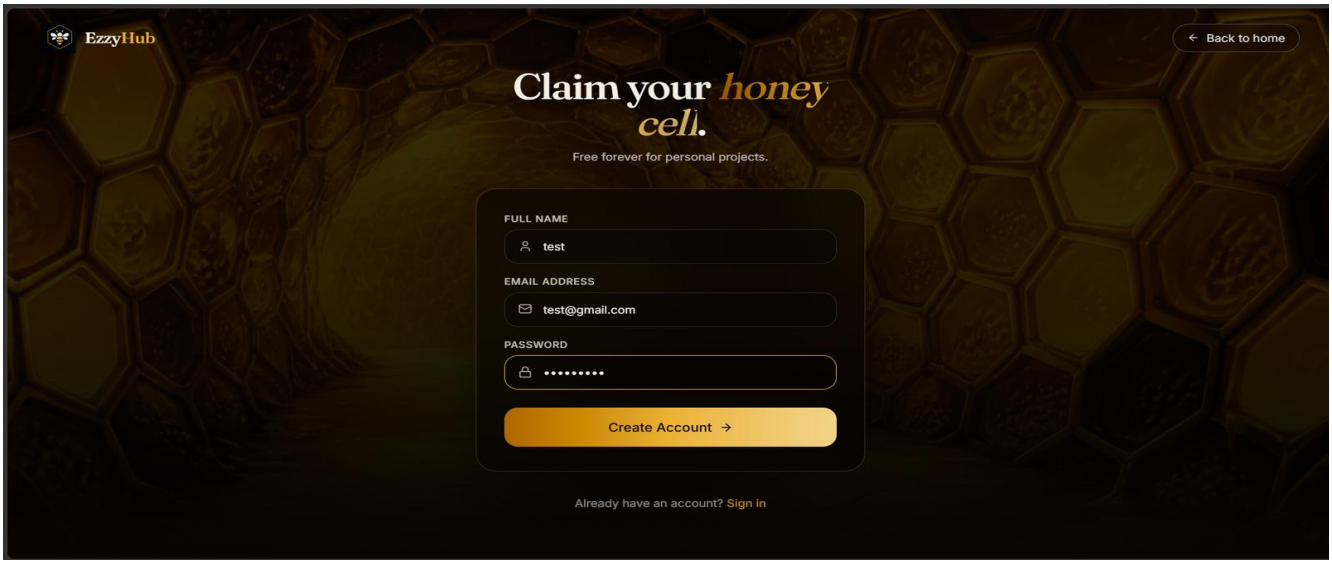
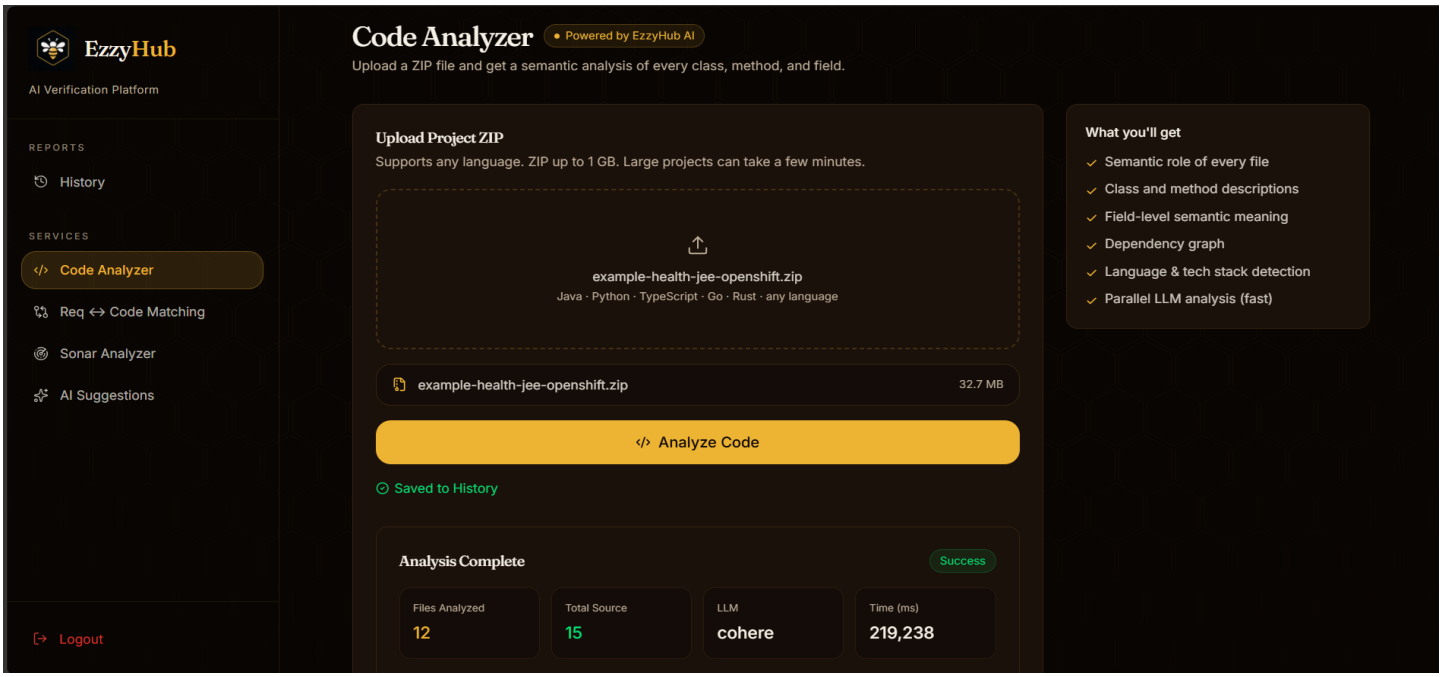
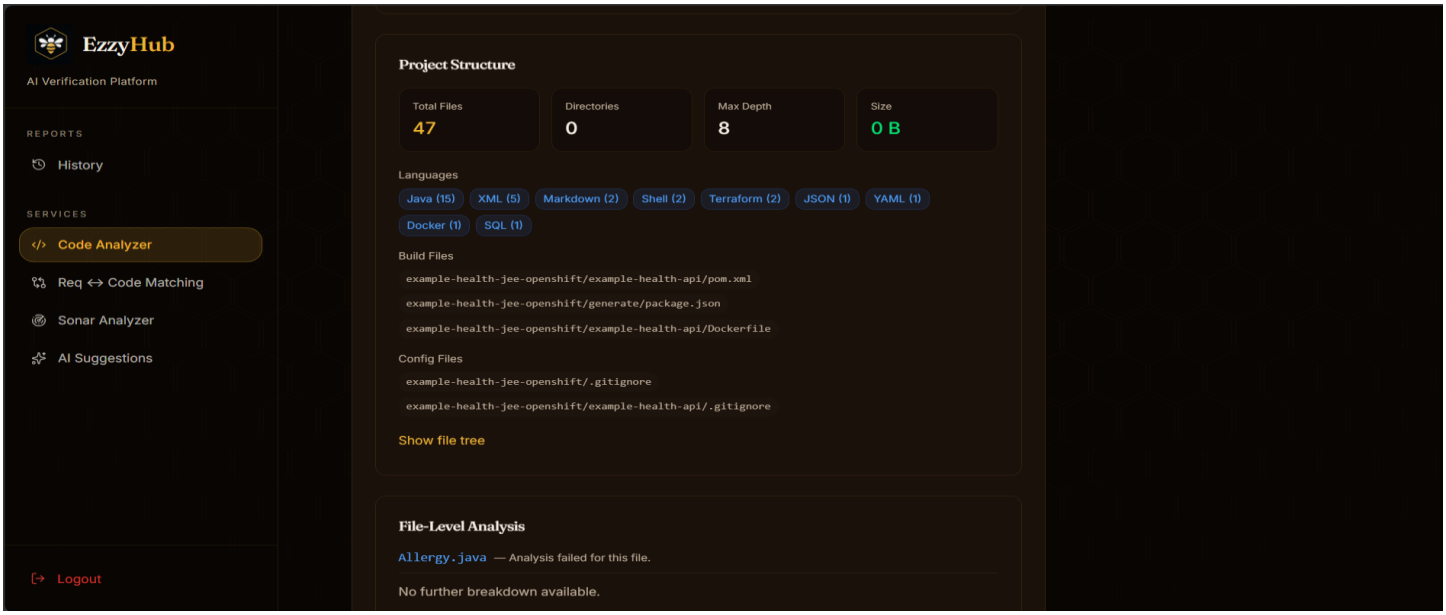


Figure 18: Sign Up Screen**5**

.3.3 Code Analyzer

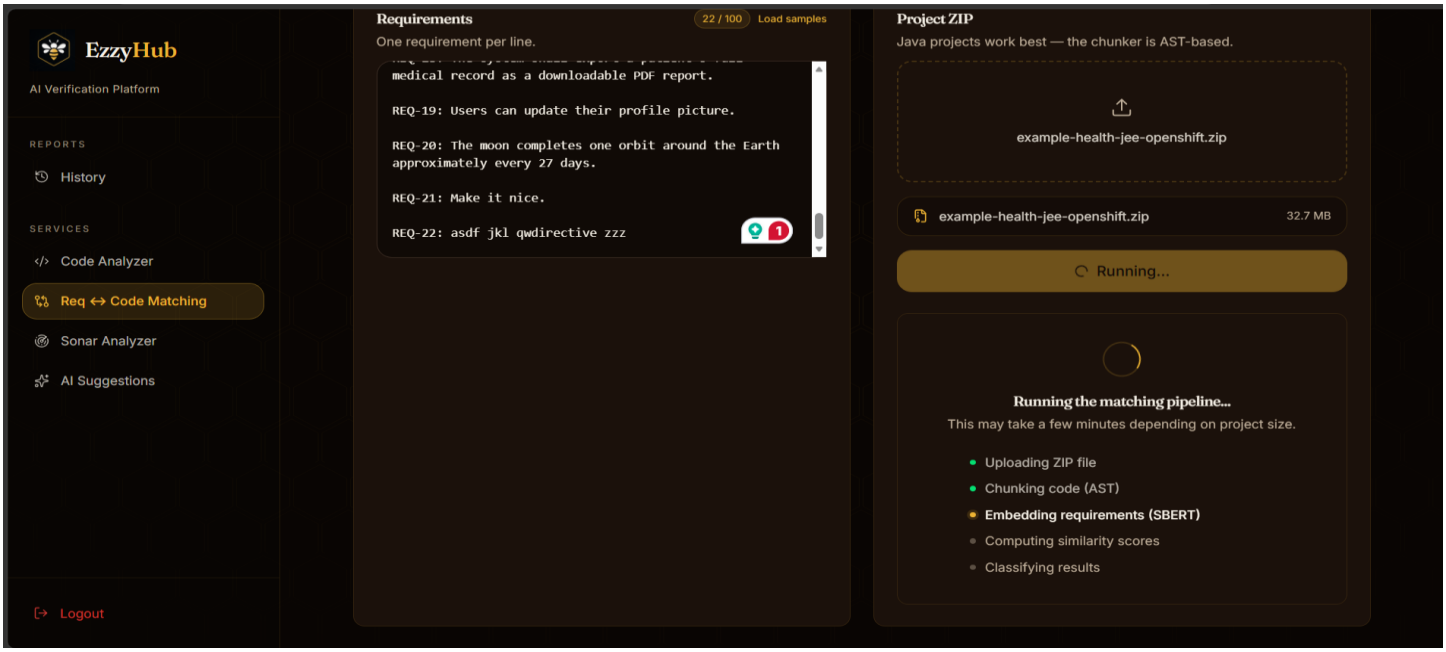

EzzyHub
AI Verification Platform
REPORTS
History
SERVICES
Code Analyzer
Req ↔ Code Matching
Sonar Analyzer
AI Suggestions
Logout
Code Analyzer
Powered by EzzyHub AI
Upload a ZIP file and get a semantic analysis of every class, method, and field.
Upload Project ZIP
Supports any language. ZIP up to 1 GB. Large projects can take a few minutes.
example-health-jee-openshift.zip
Java · Python · TypeScript · Go · Rust · any language
example-health-jee-openshift.zip
32.7 MB
Analyze Code
Saved to History
What you'll get
Semantic role of every file
Class and method descriptions
Field-level semantic meaning
Dependency graph
Language & tech stack detection
Parallel LLM analysis (fast)
Analysis Complete
Success
Files Analyzed
12
Total Source
15
LLM
cohere
Time (ms)
219,238

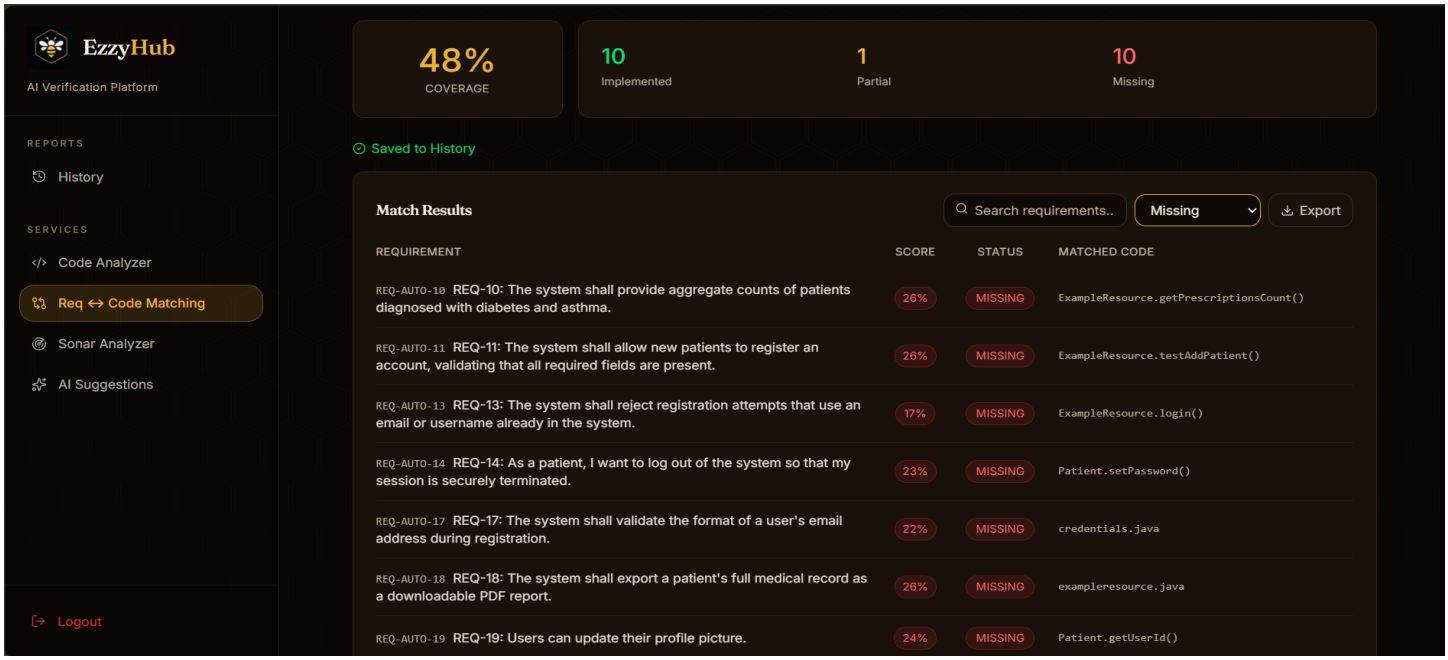

EzzyHub
AI Verification Platform
REPORTS
History
SERVICES
Code Analyzer
Req ↔ Code Matching
Sonar Analyzer
AI Suggestions
Logout
Project Structure
Total Files
47
Directories
0
Max Depth
8
Size
0 B
Languages
Java (15)
XML (5)
Markdown (2)
Shell (2)
Terraform (2)
JSON (1)
YAML (1)
Docker (1)
SQL (1)
Build Files
example-health-jee-openshift/example-health-api/pom.xml
example-health-jee-openshift/generate/package.json
example-health-jee-openshift/example-health-api/Dockerfile
Config Files
example-health-jee-openshift/.gitignore
example-health-jee-openshift/example-health-api/.gitignore
Show file tree
File-Level Analysis
Allergy.java — Analysis failed for this file.
No further breakdown available.


EzzyHub
AI Verification Platform
REPORTS
History
SERVICES
Code Analyzer
Req ↔ Code Matching
Sonar Analyzer
AI Suggestions
Logout
Credentials.java — This file defines a Credentials class for managing user authentication details.
Credentials
A class to hold user credentials.
public Credentials()
Default constructor for Credentials class.
public String getUID()
Returns the user ID.
public void setUID(String uID)
Sets the user ID.
public String getPASS()
Returns the user password.
public void setPASS(String pASS)
Sets the user password.
ExampleResource.java — This file defines a RESTful API resource class for various healthcare-related operations.
ExampleResource
A RESTful API resource class for healthcare data operations.
public Response getPatientDetails(@PathParam("patID") String patID)
Retrieves patient details by ID.

Figure 19: Code Analyzer Screens

5.3.4 Requirement-Code Matching





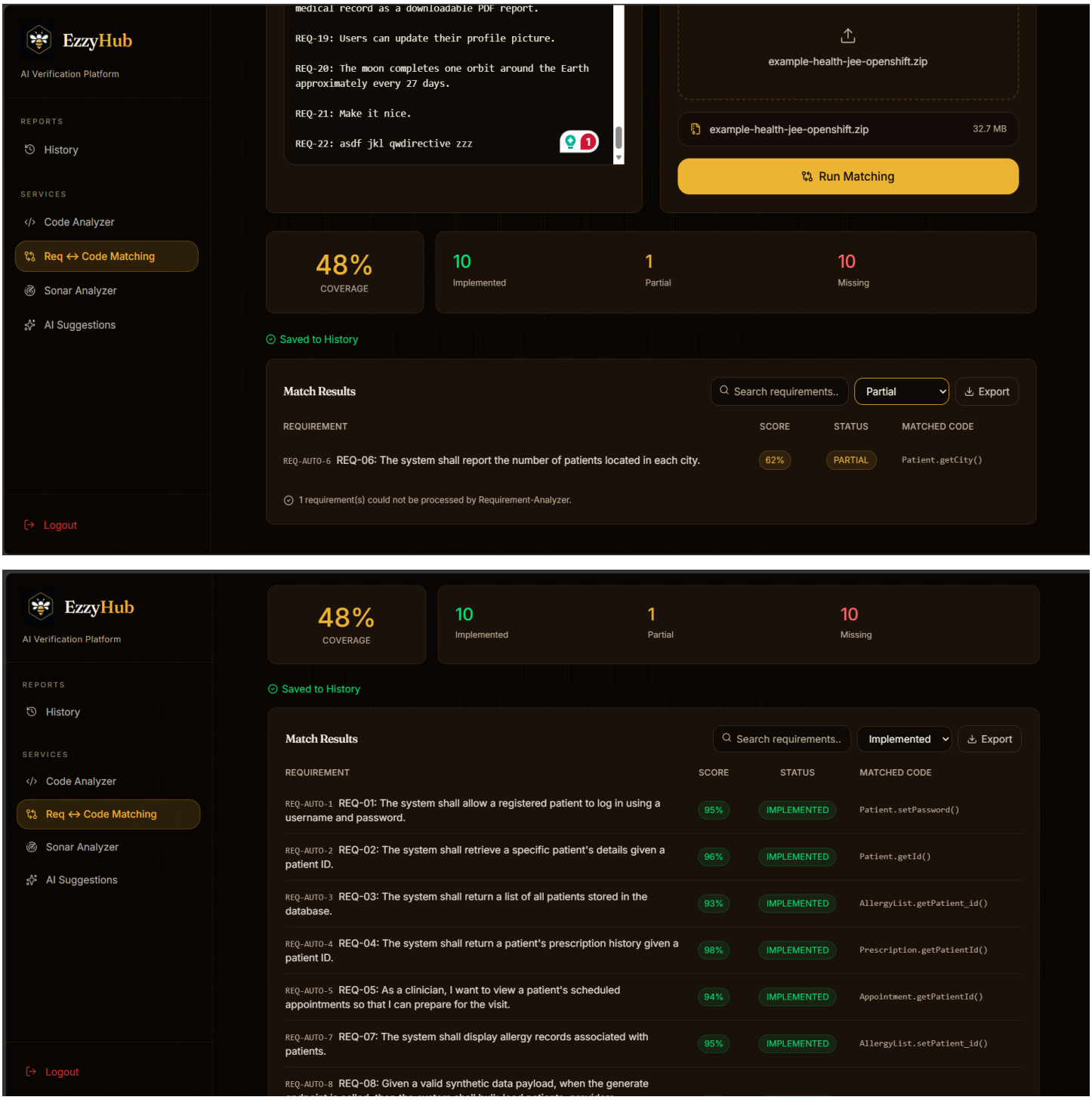
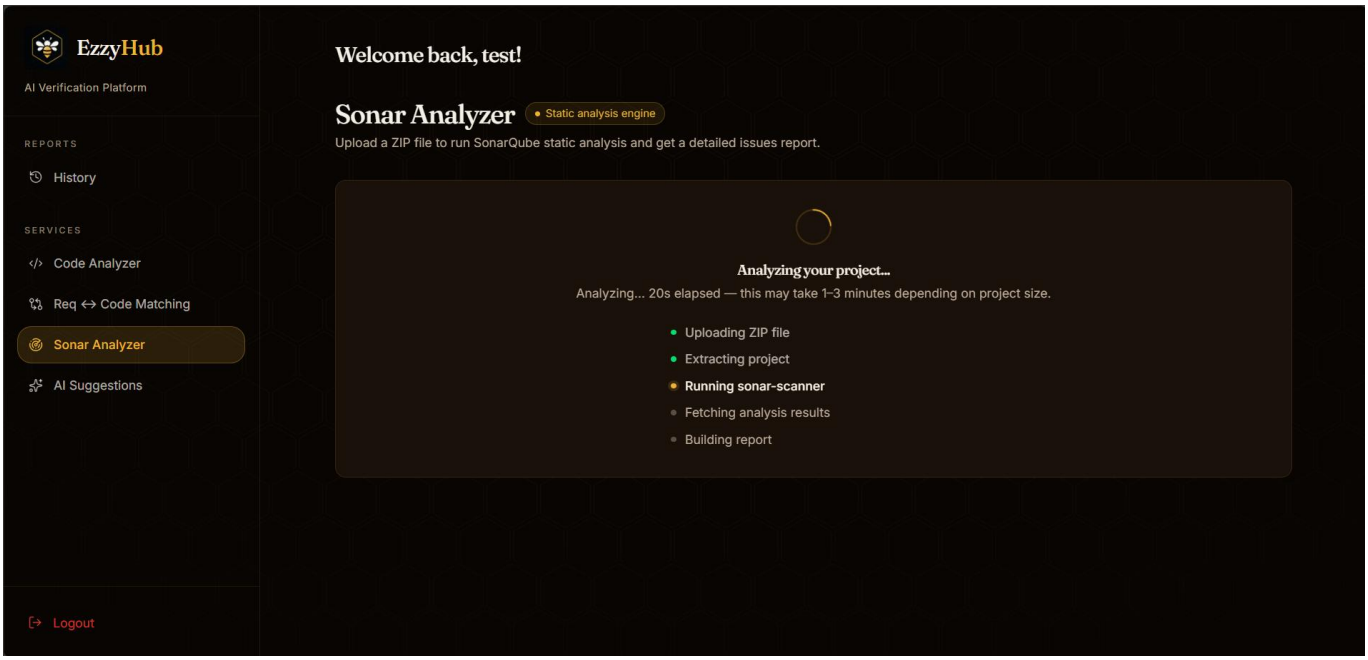


Figure 20: Requirement-Code Matching Screens

5.3.5 Sonar Analyzer



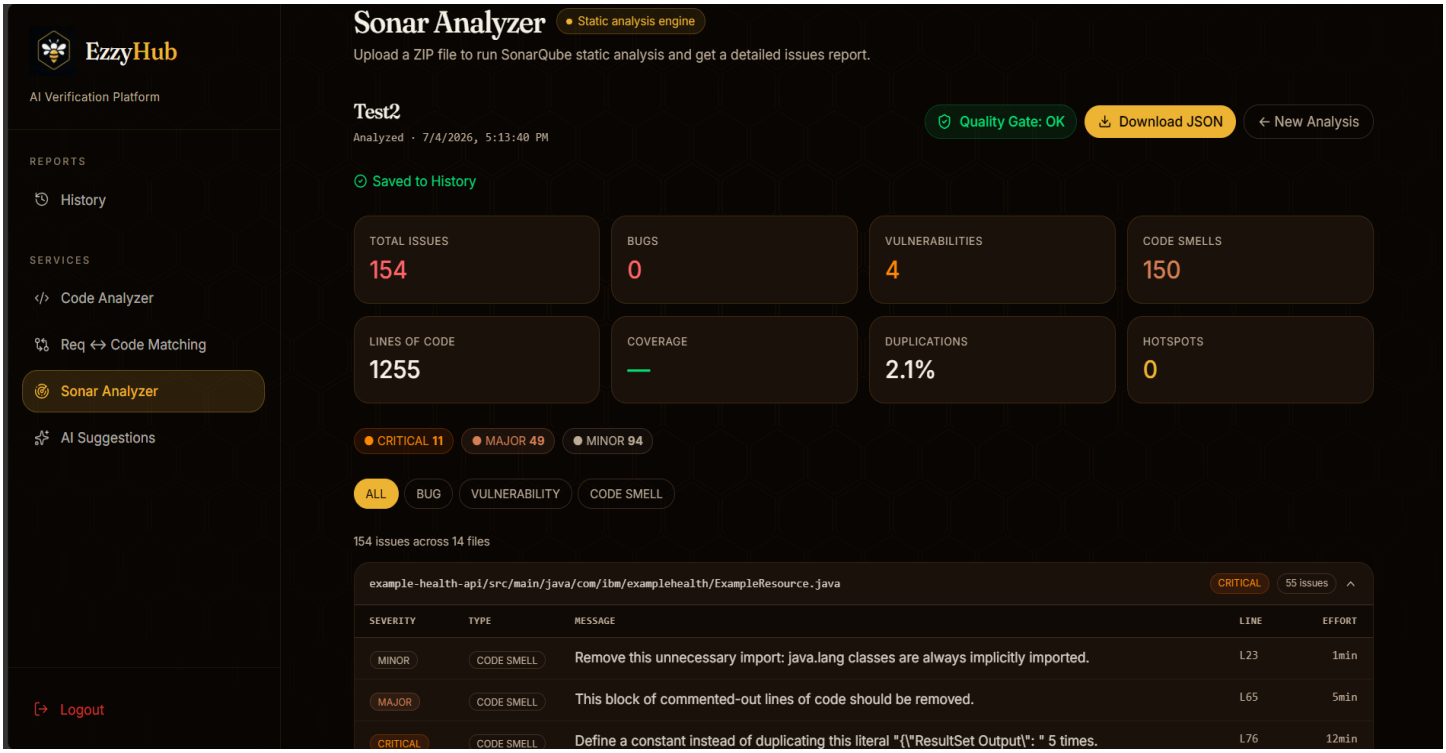
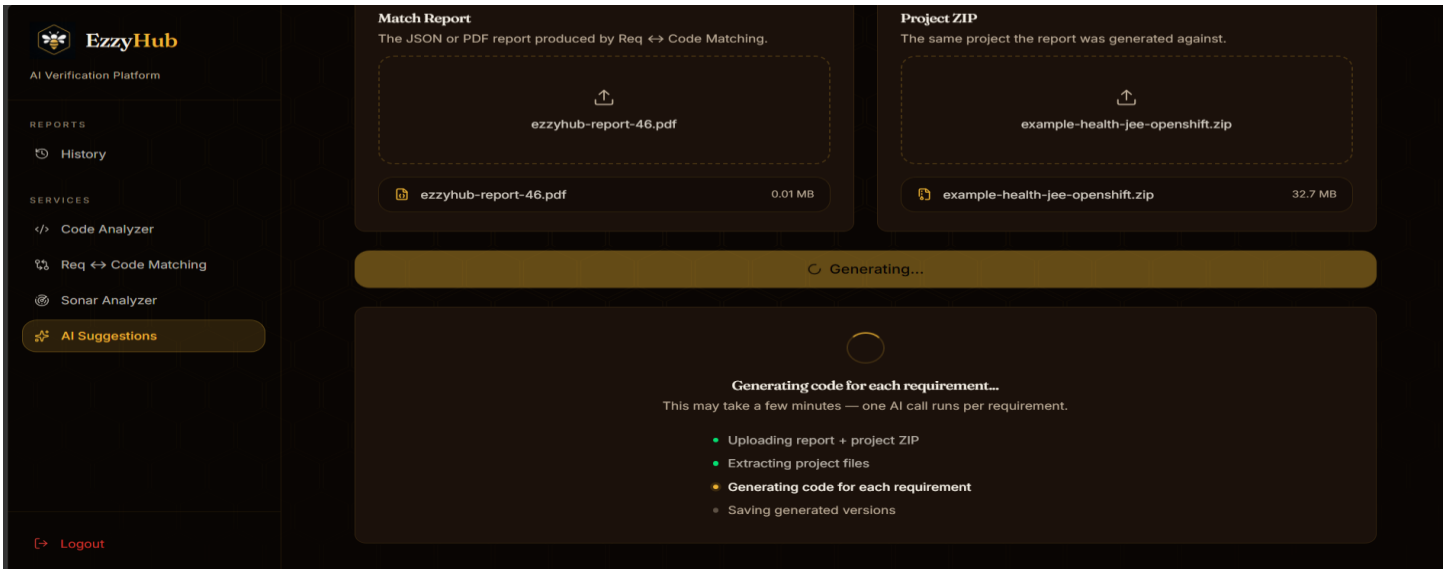
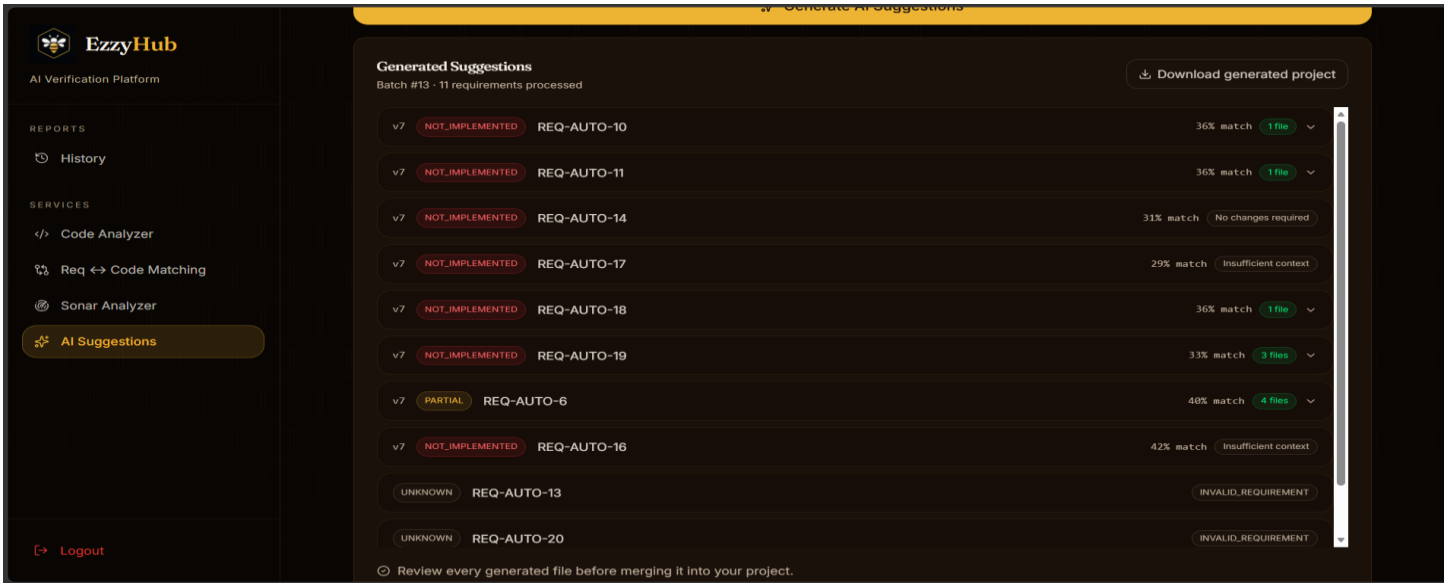


Figure 21: Sonar Analyzer Screens

5.3.6 AI Suggestions


EzzyHub
AI Verification Platform
REPORTS
History
SERVICES
Code Analyzer
Req ↔ Code Matching
Sonar Analyzer
AI Suggestions
Logout
Match Report
The JSON or PDF report produced by Req ↔ Code Matching.
ezzyhub-report-46.pdf
ezzyhub-report-46.pdf
0.01 MB
Project ZIP
The same project the report was generated against.
example-health-jee-openshift.zip
example-health-jee-openshift.zip
32.7 MB
Generating...
Generating code for each requirement...
This may take a few minutes — one AI call runs per requirement.
Uploading report + project ZIP
Extracting project files
Generating code for each requirement
Saving generated versions


EzzyHub
AI Verification Platform
REPORTS
History
SERVICES
Code Analyzer
Req ↔ Code Matching
Sonar Analyzer
AI Suggestions
Logout
Generated Suggestions
Batch #13 · 11 requirements processed
Download generated project
v7 NOT_IMPLEMENTED REQ-AUTO-10 36% match 1 file
v7 NOT_IMPLEMENTED REQ-AUTO-11 36% match 1 file
v7 NOT_IMPLEMENTED REQ-AUTO-14 31% match No changes required
v7 NOT_IMPLEMENTED REQ-AUTO-17 29% match Insufficient context
v7 NOT_IMPLEMENTED REQ-AUTO-18 36% match 1 file
v7 NOT_IMPLEMENTED REQ-AUTO-19 33% match 3 files
v7 PARTIAL REQ-AUTO-6 40% match 4 files
v7 NOT_IMPLEMENTED REQ-AUTO-16 42% match Insufficient context
UNKNOWN REQ-AUTO-13 INVALID_REQUIREMENT
UNKNOWN REQ-AUTO-20 INVALID_REQUIREMENT
Review every generated file before merging it into your project.

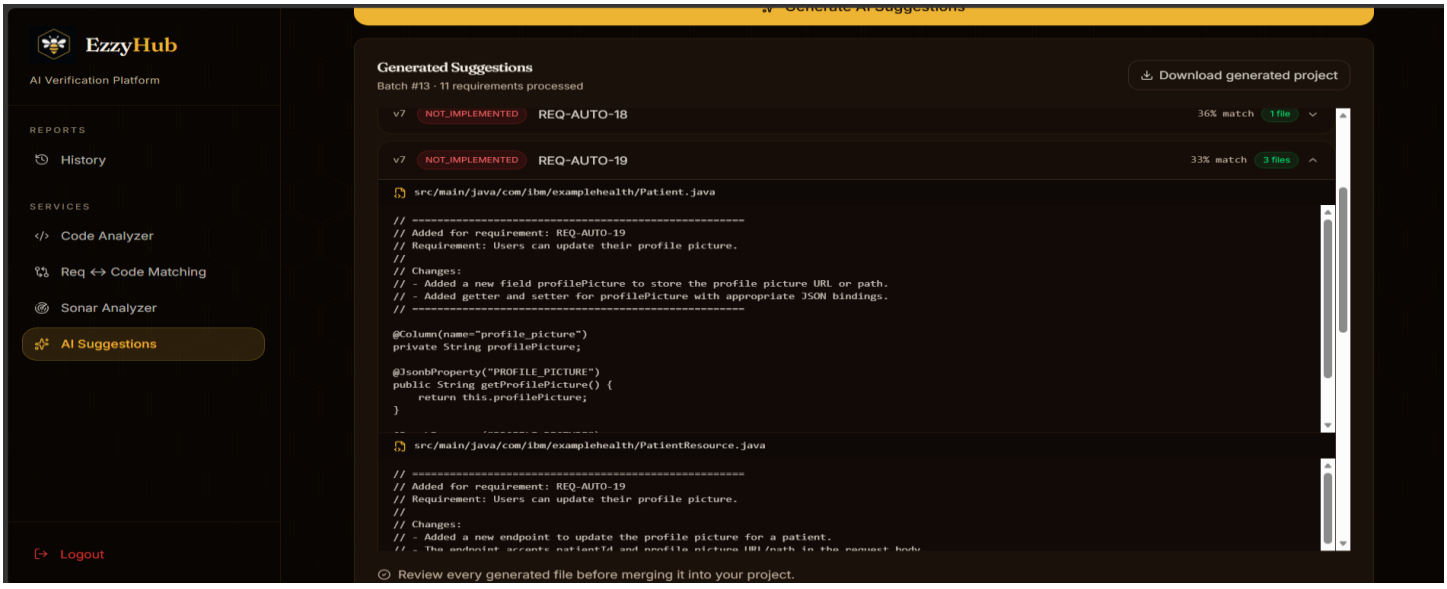

EzzyHub
AI Verification Platform
REPORTS
History
SERVICES
Code Analyzer
Req ↔ Code Matching
Sonar Analyzer
AI Suggestions
Logout
Generated Suggestions
Batch #13 · 11 requirements processed
Download generated project
v7 NOT_IMPLEMENTED REQ-AUTO-18 36% match 1 file
v7 NOT_IMPLEMENTED REQ-AUTO-19 33% match 3 files
src/main/java/com/ibm/examplehealth/Patient.java
// --
// Added for requirement: REQ-AUTO-19
// Requirement: Users can update their profile picture.
//
// Changes:
// - Added a new field profilePicture to store the profile picture URL or path.
// - Added getter and setter for profilePicture with appropriate JSON bindings.
// --
@Column(name="profile_picture")
private String profilePicture;
@JsonbProperty("PROFILE_PICTURE")
public String getProfilePicture() {
return this.profilePicture;
}
src/main/java/com/ibm/examplehealth/PatientResource.java
// --
// Added for requirement: REQ-AUTO-19
// Requirement: Users can update their profile picture.
//
// Changes:
// - Added a new endpoint to update the profile picture for a patient.
Review every generated file before merging it into your project.

Figure 22: AI Suggestions Screens

5.3.7 Report History

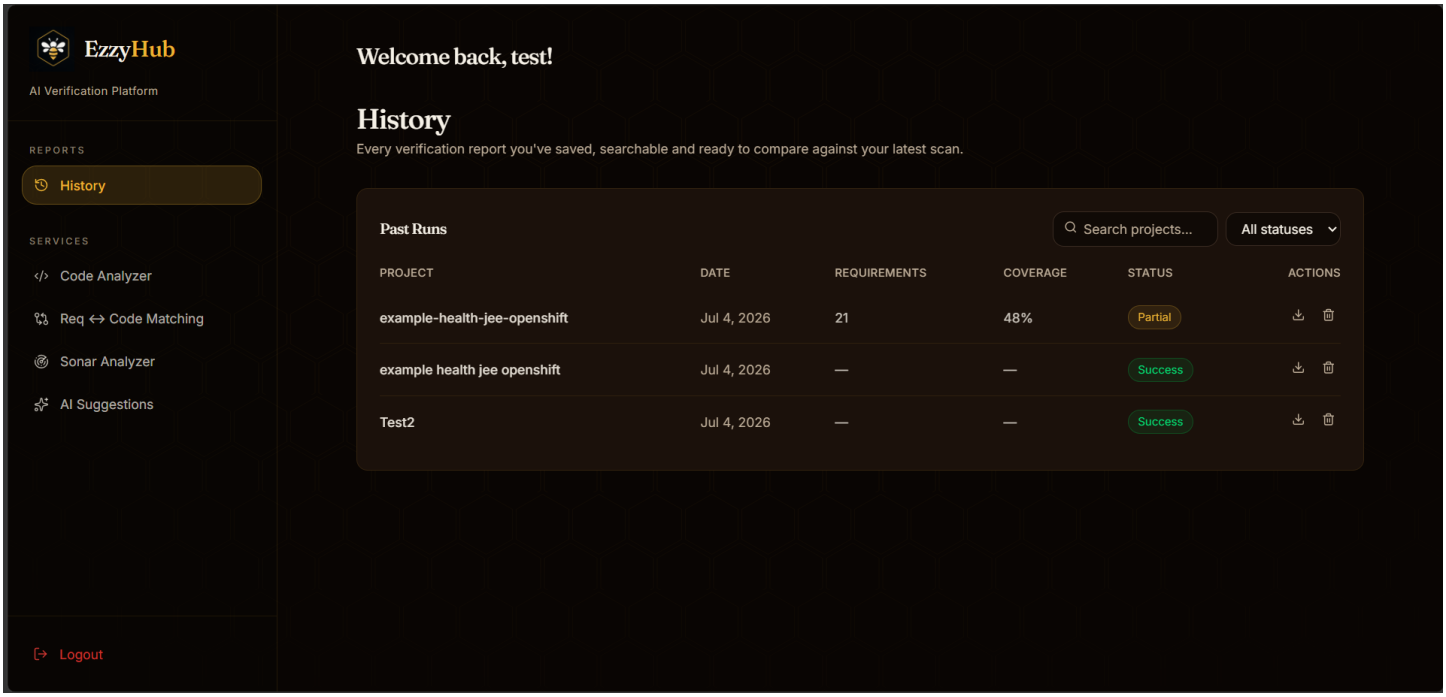


Figure 23: Report History Screen

Chapter 6: Testing and Evaluation

6.1 Testing Strategy

6.1.1 Requirements Analysis

The testing strategy adopts a multi-layered approach combining automated and manual testing across all levels of the Requirement Analyzer module.

At the unit level, JUnit 5 and Mockito were used to isolate and verify core pipeline components including requirement ID extraction, validation logic, the facade orchestrator, and the underlying NLP utility components (tokenization, keyword extraction, synonym expansion) independent of external dependencies.

At the integration level, Spring's @WebMvcTest with MockMvc validated REST endpoint behavior, while @SpringBootTest with an in-memory H2 database verified the complete analysis pipeline end-to-end.

In addition, a dedicated Postman collection was used to manually test cross-service communication between the Requirement Analyzer, Code Analyzer, and Python NLP microservices through the API Gateway, covering health checks, single and batch requirement analysis, validation edge cases, and full CRUD operations. Across both levels, a total of 56 test cases were executed with a 100% pass rate.

6.1.2 Static Analysis by sonar

The Sonar Analyzer was validated using a multi-layered strategy combining automated and manual testing, so that both internal logic and the exposed REST contract are covered.

Unit level: JUnit 5 tests verify the core model and configuration classes (**SonarIssue**, **AnalysisReport**, **CorsConfig**).

Controller level: MockMvc and Mockito validate the REST API health checks, analysis requests, input validation, job-status retrieval, and error handling without requiring a running SonarQube instance.

API level: a Postman collection manually exercises the live endpoints: uploads, polling, duplicate requests, invalid input, and unsupported methods.

Overall, **22 test cases** (14 automated + 8 Postman) were executed, achieving a **100% pass rate**.

6.1.3 Matching Service

The Matching Service was validated with a fully automated, multi-layered strategy covering its four risk areas: the classification algorithm, the pipeline orchestration around it, the REST contract, and the three outbound HTTP clients.

At the unit level, the MatchingEngine the class that decides IMPLEMENTED/PARTIAL/MISSING for every requirement was tested with formal techniques: boundary value analysis on the cosine and Jaccard similarity functions (identical, orthogonal, opposite, zero, mismatched-dimension, and null

vectors), a decision table over the classification thresholds (exactly at, just below, and far below each cut-off), a regression test for the per-file grouping bug fix, and a margin "safety-net" pair that feeds the same noisy score cluster through both the calibrated margin thresholds and the currently deployed ones. Test embeddings are constructed as exact unit vectors so every expected cosine value is mathematically precise.

At the orchestration level, the pipeline was tested with the three downstream clients mocked (Mockito) but a real MatchingEngine wired in with production thresholds, using hand-crafted orthogonal embeddings so each requirement deterministically aligns with exactly one file exercising classification, coverage math, the database fallback, the readiness gate, the fan-out cap, and the rule-based downgrades end-to-end.

At the integration level, Spring's @WebMvcTest with MockMvc validated the REST endpoint behavior (input validation, magic-byte rejection, 503/500 error translation, health reporting), and each HTTP client was tested against a real local mock web server (OkHttp MockWebServer), verifying the exact outbound request shape, response parsing, and the contracted failure behavior (fail-fast paths throw with context; best-effort paths return empty results and never throw).

The suite is fully offline and deterministic no live Python service, analyzer, database, or network connection is required. In total, 59 automated test cases were executed with a 100% pass rate.

6.1.4 Code Analyzer

Testing followed a layered strategy across each microservice, mirroring the layering of the codebase itself:

- **Unit tests** (JUnit 5 / Mockito) isolate individual components, the ZIP-extraction and project-structure logic, and the LLM client with all external dependencies (the OkHttp client, the file system) mocked, so tests run deterministically and without network access or real LLM spend.
- **Controller-level tests** (MockMvc) exercise the REST layer in isolation from the service layer, verifying that each input condition (missing file, wrong extension, capacity exceeded, downstream failure) is mapped to the correct HTTP status and response body, without starting a full server.

- **Cross-service integration tests** (Postman) exercise the system end-to-end through the API Gateway, the way the frontend actually calls it covering routing/path-rewriting, multipart forwarding, and the full request lifecycle from gateway to service and back.

This combination catches two different classes of bugs: unit and controller tests catch logic errors inside a single service, while the gateway-routed Postman suite catches integration-boundary issues the kind that unit tests can't see, such as a route forwarding to the wrong path, or two services independently attaching duplicate CORS headers to the same response.

6.1.5 AI

- The AI-Suggestions service was tested mainly through **manual integration testing using Postman** rather than automated unit tests with mocked dependencies. This approach was chosen because the service relies heavily on external and non-deterministic components such as the **OpenAI API, ZIP parsing, and PDF report processing**, which are difficult to validate meaningfully with mocks.
- Testing used **real Spring Boot projects from GitHub** and actual match reports generated by the Requirement-Analyzer service instead of synthetic data. Using real repositories exposed practical edge cases, including inconsistent file names, nested package structures, and ambiguous requirements.
- Since AI-generated responses may vary between executions, each endpoint was executed **multiple times with identical inputs** to verify output consistency and stability. Observed variations were reduced by adjusting the prompt configuration, including lowering the model temperature and fixing a seed value.
- Testing covered the **entire request lifecycle**, including:
- Authentication using valid and invalid JWT tokens.
- Uploading real ZIP archives and PDF/JSON reports via multipart requests.
- Verifying generated ZIP outputs and downloaded versions.
- Testing malformed ZIP files, prompt-injection attempts, and invalid requirements to ensure that security components such as PromptSecurityService, SensitiveDataValidator, and ZipSecurityService blocked malicious inputs before they reached the AI model or file system.

6.2 Test Cases and Results (table format)

6.2.1 Requirements Analysis

Unit & Integration Test Summary (JUnit 5 / Mockito)

Test Class	Scope	Tests	Result	Time
RequirementAnalyzerControllerTest	REST layer (MockMvc)	5	5/5	3.14s
RequirementAnalyzerFacadeTest	Facade orchestration (mocked dependencies)	6	6/6	0.14s
PipelineTest → Phase1Tests	ID extraction + basic validation	8	8/8	0.03s
PipelineTest → Phase2Tests	Tokenization / stopwords / lemmatization	6	6/6	0.03s
PipelineTest → Phase3Tests	POS tagging / keyword / synonym extraction	4	4/4	0.02s
PipelineTest → Phase4Tests	Sentence / word embeddings	4	4/4	0.05s
PipelineTest → EdgeCaseTests	Edge cases (empty input, special characters)	8	8/8	3.14s
Total		**41**	**100%**	**16.4s**

Table 3: Requirement Analyzer Unit and Integration Test Summary

API / Cross-Service Integration Test Cases (Postman)

#	Test Case	Endpoint	Expected	Actual	Status
1	Health Check	GET /requirements/health	200, UP	200, UP	Pass
2	Analyze duplicate ID update	POST /analyze	200 OK	200 OK	Pass
3	Analyze valid (AI on)	POST /analyze	201 Created	201 Created	Pass
4	Analyze skip AI validation	POST /analyze	201 Created	201 Created	Pass
5	Analyze no action verb warning	POST /analyze	201 + warning	201 + warning	Pass
6	Analyze INVALID (not a requirement)	POST /analyze	422	422	Pass
7	Analyze INVALID (too short)	POST /analyze	422	422	Pass

8	Analyze INVALID (blank)	POST /analyze	400	400	Pass
9	Batch all valid (AI on)	POST /analyze/batch	200 OK	200 OK	Pass
10	Batch skip AI	POST /analyze/batch	200 OK	200 OK	Pass
11	Batch mixed valid/invalid	POST /analyze/batch	200 OK (partial)	200 OK	Pass
12	Get All Requirements	GET /requirements	200, summary DTOs	200 OK	Pass
13	Get by ID	GET /requirements/1	200 OK	200 OK	Pass
14	Get by ID not found	GET /requirements/40	404	404	Pass
15	Delete	DELETE /requirements/1	200 OK	200 OK	Pass
Total				15/15	

Table 4: Requirement Analyzer API / Cross-Service Test Cases (Postman)

6.2.4 Static Analysis by Sonar

6.2.4.1 Unit & Controller Test Summary (JUnit 5 / Mockito)

Test Class	Scope	Tests	Result	Time
SonarControllerTest	REST API (MockMvc + Mockito)	6	6/6	523 ms
SonarIssueTest	Model Logic (getFilePath(), getters/setters)	4	4/4	41 ms
AnalysisReportTest	Model Validation (getters/setters & metrics)	3	3/3	55 ms
CorsConfigTest	Spring Configuration (CORS Bean)	1	1/1	49 ms
Total		14	14/14	≈ 668ms

Table 5: Sonar Analyzer Unit and Controller Test Summary

6.2.4.2 API / Integration Test Cases (Postman)

#	Test Case	Endpoint	Expected	Actual	Status
1	Health Check	GET /api/health	200 OK, Service = UP	200 OK, Service = UP	Pass
2	Upload Valid ZIP	POST /api/analyze	200 OK + Job ID returned	200 OK + Job ID returned	Pass
3	Poll Analysis Status	GET /api/status/{jobId}	200 OK, Status returned	200 OK, Status returned	Pass
4	Duplicate Analysis Request	POST /api/analyze	409 Conflict	409 Conflict	Pass
5	Invalid Job ID	GET /api/status/12345	404 NOT_FOUND	404 NOT_FOUND	Pass
6	Upload Invalid File (.txt)	POST /api/analyze	400 Bad Request	400 Bad Request	Pass
7	Wrong HTTP Method	POST /api/status/{jobId}	405 Method Not Allowed	405 Method Not Allowed	Pass
8	Wrong Endpoint	GET /api/healt	404 Not Found	404 Not Found	Pass
Total				8 / 8	100%

Table 6: Sonar Analyzer API / Integration Test Cases (Postman)

6.2.3 Matching Service

Unit & Integration Test Summary (JUnit 5 / Mockito / MockWebServer)

Test Class	Scope	Tests	Result	Time
MatchingEngineTest → CosineSimilarityBoundaries	Cosine similarity boundary values	6	6/6	0.01s
MatchingEngineTest → JaccardSimilarityBoundaries	Jaccard similarity boundary values	4	4/4	0.01s
MatchingEngineTest → ClassificationDecisionTable	Threshold decision table	5	5/5	0.01s
MatchingEngineTest → weighting / margins / edge cases	Score weighting, margin gate, regressions	8	8/8	0.21s

MatchingServiceTest	Pipeline orchestration (mocked clients + real engine)	7	7/7	0.10s
MatchingControllerTest	REST layer (MockMvc)	7	7/7	4.01s
PythonNlpClientTest	Python NLP client (MockWebServer)	10	10/10	0.17s
CodeAnalyzerClientTest	Code-Analyzer client (best-effort contract)	5	5/5	1.85s
RequirementAnalyzerClientTest	Requirement-Analyzer client	7	7/7	0.07s
Total		59	100%	~6.5s

Table 7: Matching Service Unit and Integration Test Summary

Representative Test Cases

#	Test Case	Expected	Actual	Status
1	Cosine: opposite vectors are clamped	0.0 (not −1)	0.0	Pass
2	Cosine: zero vector / dimension mismatch	0.0, no crash	0.0	Pass
3	Jaccard: partial overlap {a,b,c}×{b,c,d}	2/4 = 0.5	0.5	Pass
4	Combined score applies weights (cos 0.6, jac ⅓)	0.5067	0.5067	Pass
5	Score exactly at IMPLEMENTED threshold (0.44)	IMPLEMENTED	IMPLEMENTED	Pass
6	Score just below IMPLEMENTED threshold	PARTIAL	PARTIAL	Pass
7	Score just below PARTIAL threshold (0.39)	MISSING	MISSING	Pass
8	Per-file grouping: one file's chunks cannot crowd out another file	Both files in top-K	Both present	Pass
9	Calibrated margins reject a noisy above-threshold cluster	Downgraded to PARTIAL	PARTIAL	Pass
10	No code chunks → all requirements MISSING	MISSING, score 0	MISSING, 0	Pass
11	Full pipeline: 3 reqs → 1 IMPLEMENTED / 1 PARTIAL / 1 MISSING	Coverage 33.3%	33.3%	Pass

12	No inline requirements → fetched from Requirement-Analyzer DB	Matched under stored ID	As expected	Pass
13	Python not ready → readiness gate aborts	503, no downstream calls	As expected	Pass
14	Requirement count above cap	400 "Too many requirements"	As expected	Pass
15	False-friend downgrade ("secure" vs "plain text" limitation)	IMPLEMENTED → PARTIAL (capped at one level)	As expected	Pass
16	Match happy path (valid ZIP + requirements)	200 + MatchReport	200	Pass
17	Non-ZIP extension / empty file	400	400	Pass
18	ZIP-named file with non-ZIP content (magic bytes)	400 "not a valid ZIP"	400	Pass
19	Internal error hides downstream detail	Generic 500, no "port 8000" in body	As expected	Pass
20	Health endpoint reports per-dependency status	200; pythonNlp UP, reqAnalyzer DOWN	As expected	Pass
21	Batch embed body: quotes/newlines escaped correctly	Well-formed JSON	As expected	Pass
22	Python version gate (2.2.0 vs 2.1.9)	Batch used / serial fallback	As expected	Pass
23	Code-Analyzer HTTP 500	Empty map, never throws (best-effort)	As expected	Pass
24	Requirement-Analyzer analyze-batch HTTP 500	Empty list → graceful fallback to Python	As expected	Pass
Total			59/59	

Table 8: Matching Service Representative Test Cases

6.2.4 Code Analyzer

Unit & Integration Test Summary (JUnit 5 / Mockito)

Test Class	Scope	Tests	Result	Time
AnalysisControllerTest	REST layer (MockMvc) happy path, validation, capacity, error mapping	7	7/7	4.544s
ZipExtractorServiceTest	ZIP extraction & hardening (nested paths, ignored dirs, binary detection, char-limit truncation, entry cap, zip-slip rejection, corrupt archive)	9	9/9	0.202s
LLMServiceTest	Prompt assembly, JSON parsing, HTTP-error/timeout fallback, sensitive-file exclusion (mocked LLM calls)	8	8/8	1.749s
ProjectStructureServiceTest	Tree building, language distribution, entry-point detection, empty-project edge case	6	6/6	0.036s
Total		**30**	**100%**	**6.53s**

Table 9: Code Analyzer Unit and Integration Test Summary

API / Cross-Service Integration Test Cases (Postman)

#	Test Case	Endpoint	Expected	Actual	Status
1	Health Check	GET /api/code/health	200, UP	200, UP	Pass
2	Info	GET /api/code/info	200 OK	200 OK	Pass
3	Analyze valid ZIP	POST /api/code/analyze	200 OK	200 OK	Pass
4	Analyze non-ZIP filename	POST /api/code/analyze	400	400	Pass
5	Analyze empty file	POST /api/code/analyze	400	400	Pass

6	Analyze ZIP exceeds size limit	POST /api/code/analyze	400	400	Pass
7	Analyze ZIP exceeds entry-count limit	POST /api/code/analyze	400	400	Pass
8	Analyze ZIP with zip-slip entries	POST /api/code/analyze	200 OK (unsafe entries skipped)	200 OK	Pass
9	Analyze at concurrency capacity	POST /api/code/analyze	429	429	Pass
10	Analyze LLM/provider failure	POST /api/code/analyze	500, generic message	500	Pass
Total					**10/10**

Table 10: Code Analyzer API / Cross-Service Test Cases (Postman)

6.3 Performance Evaluation / Benchmarking (where applicable)

6.3.1 Requirements Analysis

A benchmarking script (Python) measured latency and throughput of the Requirement Analyzer's /analyze endpoint under various conditions (20 warm-up requests discarded; full percentile suite computed).

Scenario	Avg (ms)	p95 (ms)	p99 (ms)
WITH AI validation	253.7	372.5	677.1
WITHOUT AI validation	65.8	82.5	89.6

Table 11: Single-Requirement Latency

Concurrency	Throughput (req/s)	p95 (ms)
1	12.61	99.1
5	20.15	420.0
10	15.82	876.8

20	13.04	1831.1

Table 12: Throughput Under Concurrent Load

AI semantic validation is the dominant latency cost (~3.9× slower than skipping it), due to the external AI API round-trip. Throughput peaks at 5 concurrent clients (20.15 req/s) and degrades at higher concurrency, indicating a saturation point likely caused by blocking HTTP calls between the Java and Python services (see Section 6.4).

6.3.2 Matching Service

Unlike Section 6.3.1, where the metric of interest is latency, the critical quality attribute of the Matching Service is classification accuracy: how often its IMPLEMENTED/PARTIAL/MISSING verdicts are correct. A Python benchmark harness drives the live service over the iTrust traceability benchmark the same dataset and evaluation protocol used by Wang et al. (QRS 2022) so EzzyHub's numbers are directly comparable to published results. The dataset comprises 131 requirements, 226 Java code files, and 286 ground-truth (oracle) trace links; the harness builds the full requirement × file score matrix and computes Precision/Recall/F-measure at the standard Acceptable/Good/Excellent recall cut-points.

Cut point	**EzzyHub R / P / F**	**Wang et al. best (VSM) R / P / F**
Acceptable (R≈60%)	61.2% / 5.1% / 9.4%	64.1% / 12.9% / 21.5%
Good (R≈70%)	72.0% / 2.6% / 5.0%	70.6% / 9.1% / 16.1%
Excellent (R≈80%)	80.1% / 1.7% / 3.3%	80.4% / 4.5% / 8.6%

Table 13: Trace-Link Retrieval vs. Published Baseline (iTrust)

Metric	**Value**
True Positives (correctly IMPLEMENTED)	49 (37.4%)
False Positives	4 (3.1%)
True Negatives (correctly MISSING)	12 (9.2%)
False Negatives	29 (22.1%)
PARTIAL (indeterminate)	37 (28.2%)

Precision (TP/(TP+FP))	92.5%
Recall (TP/(TP+FN))	62.8%

Table 14: Final Status Classification vs. Oracle (131 Requirements)

On the end-user-facing metric the final verdict the service is highly precise: when it declares a requirement IMPLEMENTED it is correct 92.5% of the time, at the cost of moderate recall (62.8%). This trade-off is deliberate: a false "implemented" claim is the most damaging error in a verification report. The raw link-retrieval precision sits below the classical VSM baseline at matched recall (an expected consequence of many-to-many chunk scoring producing a dense candidate matrix), which is precisely why the pipeline does not stop at raw scores the margin gate and rule-based reconciliation convert the noisy ranking into the precise final classification of Table 2. The margin calibration itself was validated statistically before shipping: 500 simulated pure-noise trials at the real population size produced zero false IMPLEMENTED classifications. At runtime, every report is instrumented with per-stage timings; the Java matching step (an in-memory O(N×M) score matrix) completes in milliseconds, and end-to-end latency is dominated by the Python chunking and embedding stages which is why requirements are embedded in one batch HTTP call rather than N serial calls.

6.3.3 Code Analyzer

Code-Analyzer's testing to date has focused on functional correctness rather than a formal performance benchmark: unlike the Requirement Analyzer's traceability accuracy evaluation against the Wang et al. (QRS 2022) iTrust dataset, there is no equivalent labeled dataset or ground truth for "correct" code analysis output to benchmark against, so no accuracy/precision figures are reported here.

What has been tuned and verified through testing is the service's *operational* performance envelope:

- The gateway's response timeout for /api/code/analyze is set to 900s (with the frontend's own client-side timeout matched to the same value) to accommodate large-project analysis without prematurely aborting the connection.

- A bounded concurrency limit (default 10 simultaneous analyses) prevents unbounded memory/CPU growth or runaway LLM spend under load, verified by the capacity test (case 9 above) returning 429 rather than degrading silently.
- Per-file and per-batch limits (MAX_CHARS_PER_FILE, MAX_FILES_FOR_LLM, BATCH_SIZE) bound how much content is sent to the LLM per request, keeping per-request latency and cost predictable regardless of project size.

If you want an actual benchmark section here e.g. wall-clock time to analyze projects of varying size/file count, or LLM token usage per analysis that would need a dedicated measurement pass (a handful of real ZIPs of different sizes, timed end-to-end) rather than something derivable from the existing test suite. I can help set that up if you want real numbers for the thesis rather than leaving this section qualitative.

6.4 Limitations and Known Issues

6.4.1 Requirements Analysis

Concurrency bottleneck: Load testing (Section 6.3) showed throughput peaks at 5 concurrent clients (~20 req/s) then degrades at higher concurrency. This is caused by the synchronous .block() calls used in PythonNlpClient when invoking the Python microservice through Spring WebClient, which tie up a limited pool of reactive event-loop threads instead of allowing true parallel execution.

External AI dependency: Semantic validation (Step 3) depends on the AI API and adds ~3.9× latency overhead (Section 6.3). If the external API is slow or unavailable, validation quality falls back to a simple rule-based check (word count + keyword presence), which is less accurate than the LLM-based validation it replaces.

English-only NLP pipeline: Tokenization, lemmatization, and POS tagging rely on spaCy's en_core_web_sm model and a manually curated English domain synonym map. Requirements written in Arabic or other languages are not supported.

Finite domain synonym coverage: Keyword expansion (Step 11) uses a hand-curated synonym map rather than a general thesaurus (WordNet is intentionally disabled due to semantic noise). Domain

terms outside this map are not expanded, which can reduce keyword overlap for FR-7 matching on unusual or newly-introduced terminology.

6.4.2 Static Analysis by Sonar

Only ZIP archives are accepted; any other file type returns HTTP 400 Bad Request.

Only one analysis can run at a time; concurrent requests return HTTP 409 Conflict until the current run finishes.

The service depends on an external SonarQube server; if SonarQube or the Sonar Scanner is unavailable, analysis cannot complete.

Larger projects require longer processing time, since the scan runs the Sonar Scanner as an external process.

Job state is stored in memory; a service restart loses all running and completed jobs.

6.4.3 Matching Service

Margin gate disabled in deployed configuration: application.properties currently sets both margin thresholds to 0.0, which makes the anti-noise standout-margin check a no-op (the gap-based margin can never be negative, so a 0.0 bar is always cleared). A dedicated regression test loads the live properties and proves that a noisy score cluster which the calibrated margins (2.0/1.0) correctly downgrade to PARTIAL is classified IMPLEMENTED under the deployed values. Until the calibrated values are restored, requirements whose candidate pool runs uniformly hot can receive false IMPLEMENTED verdicts.

Moderate recall and a large PARTIAL band: on the iTrust benchmark (Section 6.3.2) the service misses 29 of 78 truly-implemented requirements and leaves 37 of 131 in the indeterminate PARTIAL band. Precision was deliberately prioritized, so PARTIAL results should be treated as "needs human review", not as a verdict.

Java-only and English-only: AST chunking is javalang-based, so only .java files are parsed into chunks projects in other languages produce an empty report. The downstream NLP pipeline (keywords, synonyms) supports English requirements only.

Hard dependency on the Python NLP service: the readiness gate deliberately fails closed if Python is down or still loading models, matching returns HTTP 503 and cannot proceed. The Requirement-Analyzer and Code-Analyzer integrations degrade gracefully instead, but silently reduce match quality when unavailable (no cleaned requirements, no description chunks, no false-friend reconciliation), signaled only by report warnings.

Synchronous, blocking pipeline: the entire pipeline runs on the request thread using synchronous .block() calls over Spring WebClient (the same bottleneck pattern as Section 6.4.1), and the client holds one HTTP connection open for the full run timeouts are set generously at 180–300 s for large projects and cold model starts. There is no job-ID/polling model as in the Sonar Analyzer.

Large uploads buffered in heap: the 500 MB multipart ceiling combined with the file bytes being read once for the Python client and again for the Code-Analyzer client means a single request can transiently hold roughly twice the file size in heap; concurrent large uploads risk memory exhaustion.

Thresholds calibrated on one benchmark: the score thresholds (0.44/0.39) and margin values were tuned against the iTrust dataset (226 candidate files per requirement); domains with very different vocabulary density or project sizes may require re-calibration through the configurable properties

6.4.4 Code Analyzer

- **No standalone accuracy benchmark.** Code-Analyzer's semantic output quality depends on the configured LLM provider and hasn't been measured against a labeled ground truth the way requirement-traceability was.
- **Single-instance, in-memory rate limiting.** Both the gateway's IP-based rate limiter and Code-Analyzer's own concurrency semaphore are process-local: they reset on restart and don't coordinate across multiple running instances, so they wouldn't hold under a horizontally-scaled deployment without a shared store (e.g. Redis).
- **LLM-dependent availability.** Because analysis calls out to a third-party LLM provider, service behavior (latency, and in some cases the semantic-analysis quality) is bounded by that provider's own uptime and rate limits, not fully within the service's own control.

- **No live end-to-end HTTP load testing performed.** Verification for concurrency/timeout behavior relied on unit-level simulation (mocked slow calls, forced semaphore exhaustion) rather than a real sustained-load test against a running instance, since the development sandbox used for the most recent hardening pass couldn't open its own loopback sockets.
- **Residual prompt-injection surface.** Analyzed source content (including code comments) is passed to the LLM as part of the analysis prompt; a maliciously crafted comment could attempt to influence the LLM's output. This is inherent to any LLM-based static analysis tool and is mitigated downstream analysis output is only ever rendered as escaped display text, never executed but isn't eliminated at the source.

Chapter 7: Conclusion and Future Work

7.1 Summary of Achievements

- **Integrated Platform**: Microservices-based system with full flow from requirement entry and ZIP upload to downloadable reports.
- **Requirement Understanding**: Four-stage pipeline converts free-text requirements into structured, comparable forms.
- **High Accuracy & Speed**: Matching Service reached **92.5% precision** on iTrust dataset; Requirement Analyzer processes one requirement in ~66 ms, handling ~20 requests/sec.
- **AI Remediation**: Generates candidate source code with explanations to fix missing or partial requirements.
- **Static Analysis**: SonarQube pipeline detects bugs, vulnerabilities, and code smells with detailed reports.
- **Built-in Security**: Zip-slip protection, JWT authentication, input whitelisting, externalized secrets, patched dependencies.
- **Layered Testing**: Unit, controller, and integration tests validated all services and system-level flows.

7.2 Lessons Learned

- **Integration is where projects live or die:** every service worked alone; the real bugs appeared at the seams, like the Java–Python calls (buffer sizes, models still loading, timeouts).
- **AI isn't magic:** LLMs fail quietly and creatively (malformed JSON, mid-demo outages, confident wrong answers), so every AI call ended up wrapped in validation and fallbacks, each safeguard paid for with a night of debugging.
- **Silent bugs are the scariest:** the missing API key that turned embeddings into zero vectors didn't crash anything, it just made every score meaningless; better to fail loudly than succeed falsely.
- **Benchmarking humbled us:** "it seems to work" versus "we can prove how well it works" turned out to be very different sentences.
- **Microservices are an organizational tool too:** parallel work, but no one person knew the whole system, so contracts and communication became survival tools.
- **Respect for scope:** a smaller finished system you can defend beats an ambitious one that almost works.

7.3 Future Enhancements

- **Verification phase for AI-suggested code:** add an execution stage that takes the code generated by the AI suggestion service, then builds and runs it in an isolated environment to confirm it compiles and behaves correctly before it is delivered to the user.
- **GitHub repository input:** accept a GitHub repository URL as a code source alongside the current ZIP upload, so projects can be verified directly from version control.
- **Higher output accuracy:** improve the accuracy of the system's results, particularly requirement-to code matching (better embeddings and threshold calibration) and the precision of the generated reports.

  References

References (IEEE format)

[1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP-IJCNLP, Hong Kong, 2019, pp. 3982–3992.
[2] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet Challenge: Evaluating the State of Semantic Code Search," arXiv:1909.09436, 2019.
[3] M. Honnibal and I. Montani, "spaCy: Industrial-Strength Natural Language Processing," 2017. [Online]. Available: https://spacy.io
[4] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python. Sebastopol, CA, USA: O'Reilly Media, 2009.
[5] G. A. Miller, "WordNet: A Lexical Database for English," Commun. ACM, vol. 38, no. 11, pp. 39–41, 1995.[6] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packaged Resources to Advance General Chinese Embedding," arXiv:2309.07597, 2023.
[7] A. Vaswani et al., "Attention Is All You Need," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Long Beach, CA, USA, 2017, pp. 5998–6008.
[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, Minneapolis, MN, USA, 2019, pp. 4171–4186.
[9] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in Proc. NeurIPS, 2020, pp. 5776–5788.
[10] C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," J. Mach. Learn. Res., vol. 21, no. 140, pp. 1–67, 2020.
[11] T. Nguyen et al., "MS MARCO: A Human Generated MAchine Reading COmprehension Dataset," arXiv:1611.09268, 2016.
[12] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," arXiv:1301.3781, 2013.
[13] T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," in Proc. EMNLP: System Demonstrations, 2020, pp. 38–45.
[14] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Proc. NeurIPS, Vancouver, BC, Canada, 2019, pp. 8024–8035.
[15] G. Salton, A. Wong, and C.-S. Yang, "A Vector Space Model for Automatic Indexing," Commun. ACM, vol. 18, no. 11, pp. 613–620, 1975.
[16] P. Jaccard, "The Distribution of the Flora in the Alpine Zone," New Phytologist, vol. 11, no. 2, pp. 37–50, 1912.
[17] G. Antoniol, G. Canfora, G. Casazza, A. De Lucia, and E. Merlo, "Recovering Traceability Links between Code and Documentation," IEEE Trans. Softw. Eng., vol. 28, no. 10, pp. 970–983, 2002.
[18] J. Cleland-Huang, O. C. Z. Gotel, J. Huffman Hayes, P. Mäder, and A. Zisman, "Software Traceability: Trends and Future Directions," in Proc. Future of Software Engineering (FOSE), Hyderabad, India, 2014, pp. 55–69.
[19] J. H. Hayes, A. Dekhtyar, and S. K. Sundaram, "Advancing Candidate Link Generation for Requirements Tracing: The Study of Methods," IEEE Trans. Softw. Eng., vol. 32, no. 1, pp. 4–19, 2006.
[20] VMware, "Spring Boot Reference Documentation," 2024. [Online]. Available: https://spring.io/projects/spring-boot

[21] C. Thunes, "javalang: Pure Python Java Parser and Tools," 2020. [Online]. Available: https://github.com/c2nes/javalang
[22] S. Ramírez, "FastAPI," 2018. [Online]. Available: https://fastapi.tiangolo.com
[23] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.
[24] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: https://owasp.org/Top10/
[25] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2023. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/
[26] M. Souppaya, J. Morello, and K. Scarfone, "Application Container Security Guide," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST Special Publication 800-190, Sep. 2017.